

第二讲 最朴素的机器学习预测算法 ---K-近邻法

主要内容

- 继续讨论**案例二**的**回归预测**问题
 - 从局部平均到K-近邻法
 - 讨论K-近邻法的回归预测和非线性特征
 - 讨论K-近邻法的几个核心问题
- 继续讨论**案例三**的**多分类预测**问题
 - 讨论K-近邻法的分类预测和非线性特征
 - 从案例模型评价引申出数据预测中的几个重要问题
- K-近邻法的**思考**和**适用性**探讨

案例二：共享单车日租赁量的预测：回归预测

• 案例二：共享单车日租赁量 (Casual/Registered/count) 的预测：回归预测

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- Season – 1:spring 2:summer 3:fall 4:winter
- Holiday – whether the day is considered a holiday
- Workingday: whether the day is neither a weekend nor holiday
- Weather -
 - 1: Clear, Few clouds, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- Temp - temperature in Celsius
- Atemp - "feels like" temperature in Celsius
- Humidity - relative humidity
- Windspeed - wind speed
- Casual – number of non-registered user rentals initiated
- Registered - number of registered user rentals initiated
- count - number of total rentals

- 案例二的分析：一般线性回归模型
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)

$$casual = \beta_0 + \beta_1 temp + \beta_2 atemp + \beta_3 humidity + \beta_4 windspeed$$

$$registered = \beta_0 + \beta_1 temp + \beta_2 atemp + \beta_3 humidity + \beta_4 windspeed$$

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

```
X = data.iloc[:, [4,5,6,7]]
modellR = LM.LinearRegression()
for Y in ['casual', 'registered']:
    y = data[Y]
    modellR.fit(X, y)
    print('\n对%s的预测: R方=%f;MAE=%f'%(Y, modellR.score(X, y), mean_absolute_error(y, modellR.predict(X))))
```

仅采用数值型变量！！

对casual的预测: R方=0.324223;MAE=26.466792

对registered的预测: R方=0.168645;MAE=100.595709

	casual	registered	count
count	9550.000000	9550.000000	9550.000000
mean	36.957906	159.184817	196.142723
std	50.270002	151.809675	181.981750
min	0.000000	0.000000	1.000000
25%	4.000000	41.000000	47.000000
50%	18.000000	122.000000	150.000000
75%	50.000000	227.000000	289.000000
max	362.000000	886.000000	977.000000

- 案例二的分析：一般线性回归模型
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)
 - **调整模型**：将分类变量(转为虚拟变量形式后)引入回归模型

season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	season_2	season_3	season_4	weather_2	weather_3
1	0	0	2	9.84	12.880	75.0	6.0032	0	1	1	0	0	0	1	0
1	0	0	1	15.58	19.695	76.0	16.9979	12	24	36	0	0	0	0	0
1	0	0	1	14.76	16.665	81.0	19.0012	26	30	56	0	0	0	0	0
1	0	0	1	17.22	21.210	77.0	19.0012	29	55	84	0	0	0	0	0
1	0	0	2	18.86	22.725	72.0	19.9995	47	47	94	0	0	0	1	0

- 结果解读：较前个模型预测误差有所降低

调整后的线性回归模型

对casual的预测：R方=0.452398;MAE=25.435649

对registered的预测：R方=0.215822;MAE=97.870740

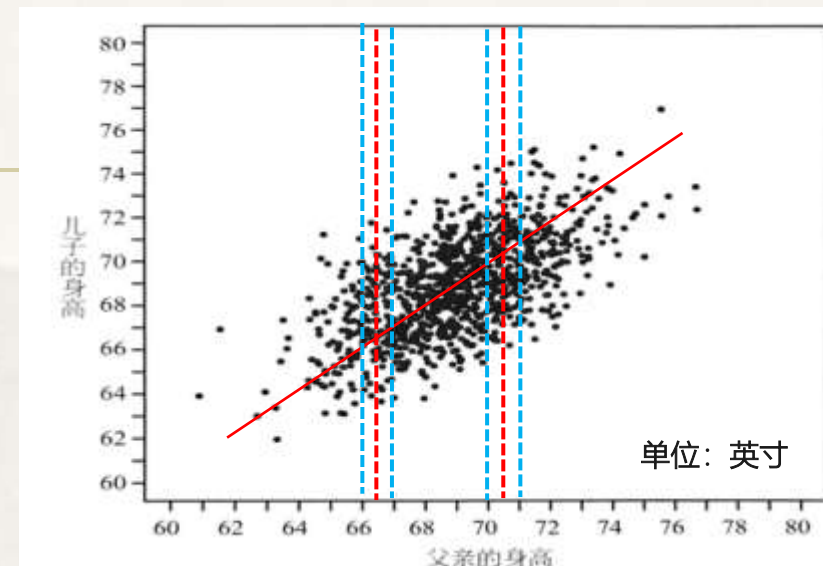
前个模型

对casual的预测：R方=0.324223;MAE=26.466792

对registered的预测：R方=0.168645;MAE=100.595709

- 预测结果仍不理想的原因是什么？
 - **非线性关系！** 如何处理？

- 再看得到回归线的策略
 - 局部平均：由众多局部均值点平滑连接得到
 - 局部平均→加权局部平均→K-近邻法
 - 最朴素的机器学习策略
- K-近邻法实现回归预测
 - 为预测样本观测 X_0 输出变量 y_0 的取值：
 - 在数据集中找到与 X_0 相似的若干个(如 K 个)样本观测，记为 $(X_1, X_2, \dots, X_K)$ 。这些样本观测称为 X_0 的近邻：对应局部
 - 对近邻 $(X_1, X_2, \dots, X_K)$ 的输出变量 $(y_1, y_2, \dots, y_K)$ ，依函数 f 计算诸如均值或加权均值等，并以此作为样本观测 X_0 输出变量取值 y_0 的预测值 $\hat{y}_0$



回归预测：

$$\hat{y}_0 = f(y_1, y_2, \dots, y_K) = \frac{1}{K} \sum_{i=1}^K y_i$$

$$\hat{y}_0 = f(y_1, y_2, \dots, y_K) = \frac{1}{\sum w_i} \left(\sum_{i=1}^K w_i y_i \right)$$

- 案例二的分析：**一般线性回归分析**→**K-近邻法**
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)
 - 指定找**1**个近邻

```
1 X = data[['temp', 'atemp', 'humidity', 'windspeed']]
2 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=1)
3 for Y in ['casual', 'registered']:
4     y = data[Y]
5     modelKNN.fit(X, y)
6     print('\n对%s的预测: R方=%f;MAE=%f' % (Y, modelKNN.score(X, y), mean_absolute_error(y, modelKNN.predict(X))))
```

仅采用数值型输入变量!

对casual的预测: R方=0.516859;MAE=14.046114

对registered的预测: R方=0.200637;MAE=66.381040

调整后的线性回归模型

对casual的预测: R方=0.452398;MAE=25.435649

对registered的预测: R方=0.215822;MAE=97.870740

- 问:
 - 为什么K-近邻法进一步降低了预测误差?
 - 如何理解K-近邻法的**非线性**特点?

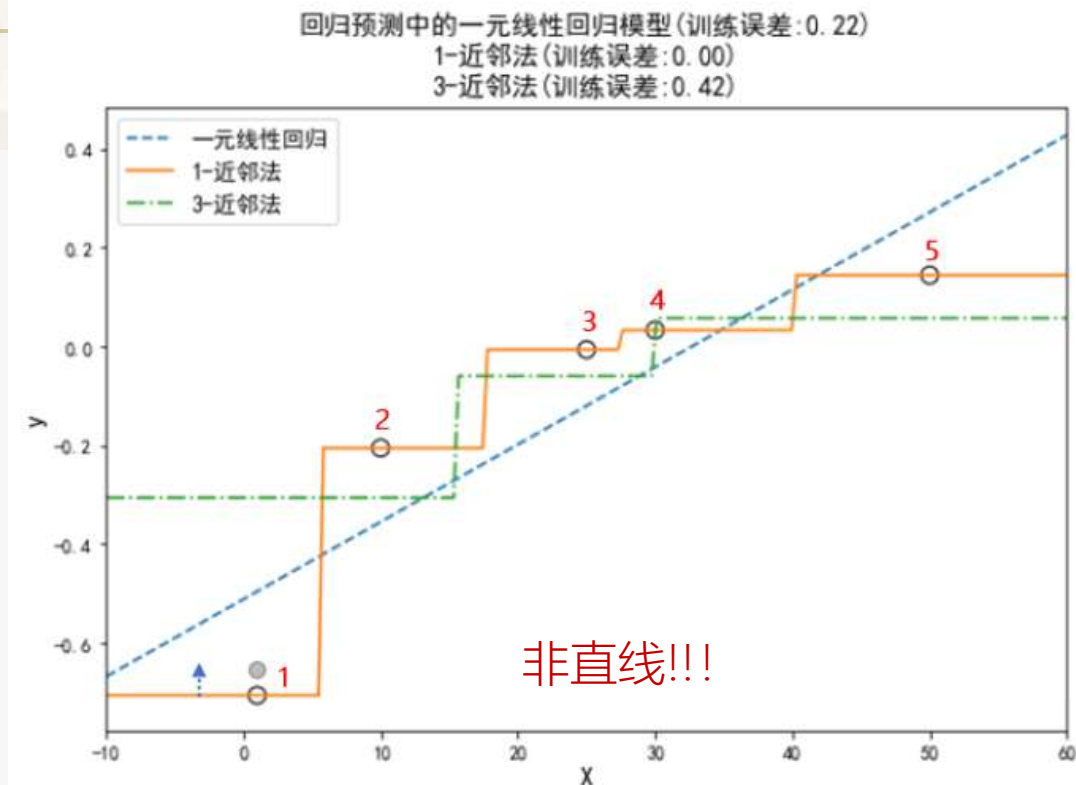
- 回归预测中的K-近邻法：非线性建模
 - Python模拟和启示：认识K-近邻的回归线

Chapter5-1-1.ipynb

```

1 N = 5
2 X = np.array([1, 10, 25, 30, 50])
3 np.random.seed(123)
4 y = 0.5 * np.log10(2 * X) + np.random.normal(10, 10)
5 plt.figure(figsize=(9, 6))
6 plt.scatter(X, y, edgecolors='black', marker='o', c='', s=80)
7 plt.xlim(xmax=60, xmin=-10)
8 X = X.reshape(N, 1)
9 modelLR = LM.LinearRegression()
10 modelLR.fit(X, y)
11 model1NN = neighbors.KNeighborsRegressor(n_neighbors=1)
12 model1NN.fit(X, y)
13 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=3)
14 modelKNN.fit(X, y)
15 X0 = np.linspace(-10, 60, 200).reshape(200, 1)
16 plt.plot(X0, modelLR.predict(X0), linestyle='--', label='一元线性回归')
17 plt.plot(X0, model1NN.predict(X0), linestyle='-', label='1-近邻法')
18 plt.plot(X0, modelKNN.predict(X0), linestyle='-.', label='3-近邻法')
19 plt.title("回归预测中的一元线性回归模型(训练误差: %.2f) \n 1-近邻法(训练误差: %.2f) \n 3-近邻法(训练误差: %.2f)" % (1-modelLR.score(X,
20 plt.xlabel("X", fontsize=14)
21 plt.ylabel("y", fontsize=14)
22 plt.legend(fontsize=12)

```

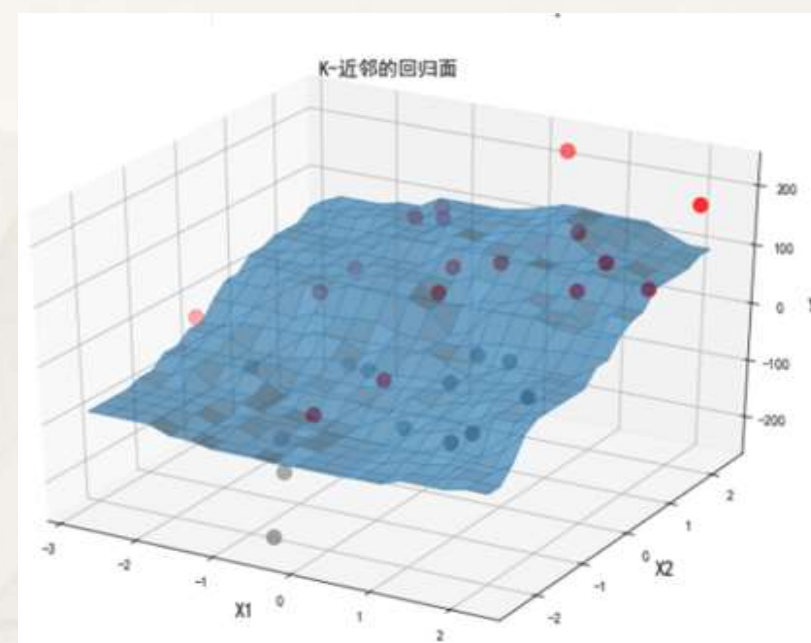
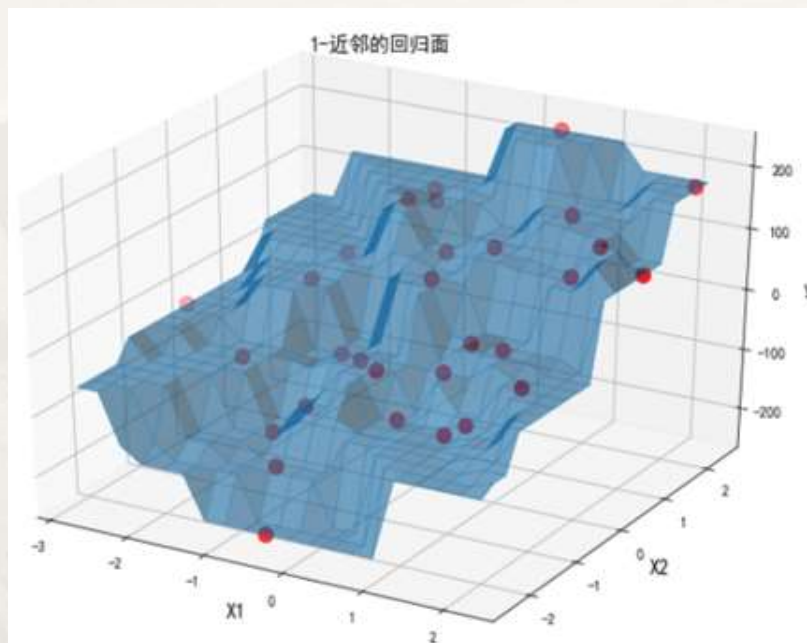
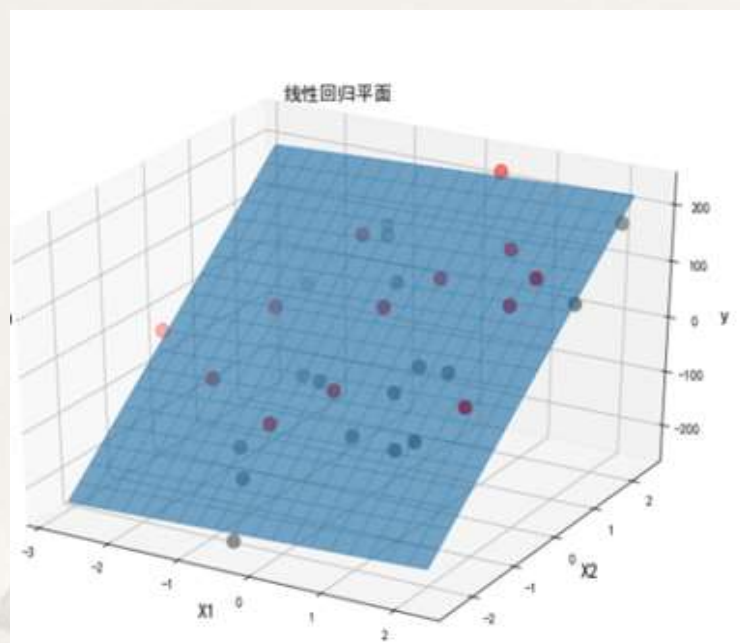


- 回归预测中的K-近邻法：非线性建模
 - Python模拟和启示：认识K-近邻的回归面

Chapter5-1-1.ipynb

```
1 N = 30
2 X, y = make_regression(n_samples=N, n_features=2, n_informative=2, noise=20, random_state=123)
3 modelLR = LM.LinearRegression()
4 modelLR.fit(X, y)
5 print('线性回归模型的训练误差: %.3f' % (1 - modelLR.score(X, y)))
6 model1NN = neighbors.KNeighborsRegressor(n_neighbors=1)
7 model1NN.fit(X, y)
8 print('1-近邻法的训练误差: %.3f' % (1 - model1NN.score(X, y)))
9 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=10)
10 modelKNN.fit(X, y)
11 print('K-近邻法的训练误差: %.3f' % (1 - modelKNN.score(X, y)))
12 X01, X02 = np.meshgrid(np.linspace(X[:, 0].min(), X[:, 0].max(), 20), np.linspace(X[:, 1].min(), X[:, 1].max(), 20))
13 X0 = np.hstack((X01.reshape(400, 1), X02.reshape(400, 1)))
14 def Myplot(y0hat, title, yhat):
15     ax = Axes3D(plt.figure(figsize=(9, 6)))
16     ax.scatter(X[y >= yhat, 0], X[y >= yhat, 1], y[y >= yhat], c='red', marker='o', s=100)
17     ax.scatter(X[y < yhat, 0], X[y < yhat, 1], y[y < yhat], c='grey', marker='o', s=100)
18     ax.plot_surface(X01, X02, y0hat.reshape(20, 20), alpha=0.6)
19     ax.plot_wireframe(X01, X02, y0hat.reshape(20, 20), linewidth=0.5)
20     ax.set_title(title, fontsize=14)
21     ax.set_xlabel('X1', fontsize=14)
22     ax.set_ylabel('X2', fontsize=14)
23     ax.set_zlabel('y', fontsize=14)
24 Myplot(y0hat=modelLR.predict(X0), title="线性回归平面", yhat=modelLR.predict(X))
25 Myplot(y0hat=model1NN.predict(X0), title="1-近邻的回归面", yhat=model1NN.predict(X))
26 Myplot(y0hat=modelKNN.predict(X0), title="K-近邻的回归面", yhat=modelKNN.predict(X))
```

- 回归预测中的K-近邻法：非线性建模
 - Python模拟和启示：认识K-近邻的回归面



非平面!!!

- 问：
 - 通常认为与一般线性回归模型相比，K-近邻法更“宽松”，如何理解？
 - K-近邻预测基于了怎样的最朴素的信念？
 - K-近邻法涉及哪些核心问题？

K-近邻法的核心问题

- 第一，依怎样的标准测度样本观测点与 X_0 的近邻关系，即如何确定 $(X_1, X_2, \dots, X_K)$
- 第二，需要找到 X_0 的几个近邻
- 第三，如何确定 $(X_1, X_2, \dots, X_K)$ 的权重

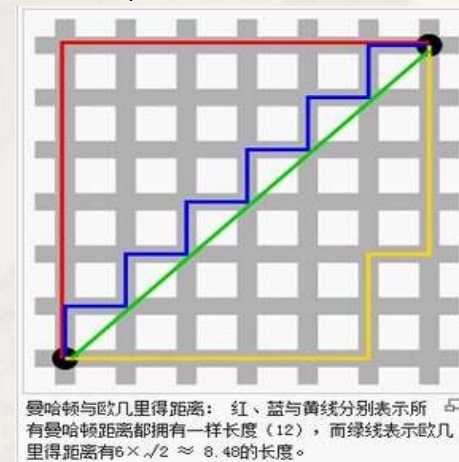
- 核心问题1：如何确定 X_0 的近邻($X_1, X_2, \dots, X_K$)？
 - 将样本观测点看成为 p 维(p 个数值型输入变量) $\mathbb{R}^p$ 中的点，在 $\mathbb{R}^p$ 中定义某种距离，并依此测度样本观测点与 X_0 的近邻关系。常用的距离：
 - 闵可夫斯基距离→欧氏距离

$$d_{Minkowski}(X_i, X_j) = \sqrt[m]{\sum_{k=1}^p |X_{ik} - X_{jk}|^m}$$

$$d_{Euclidean}(X_i, X_j) = \sqrt{\sum_{k=1}^p (X_{ik} - X_{jk})^2}$$

- 曼哈顿距离 $d_{Manhattan}(X_i, X_j) = \sum_{k=1}^p |X_{ik} - X_{jk}|$

- 夹角余弦相似度(距离) $d_{Cosine}(X_i, X_j) = \frac{\sum_{k=1}^p (X_{ik} X_{jk})}{\sqrt{\sum_{k=1}^p X_{ik}^2} \sqrt{\sum_{k=1}^p X_{jk}^2}}$
 - 余弦越大两向量的夹角越小

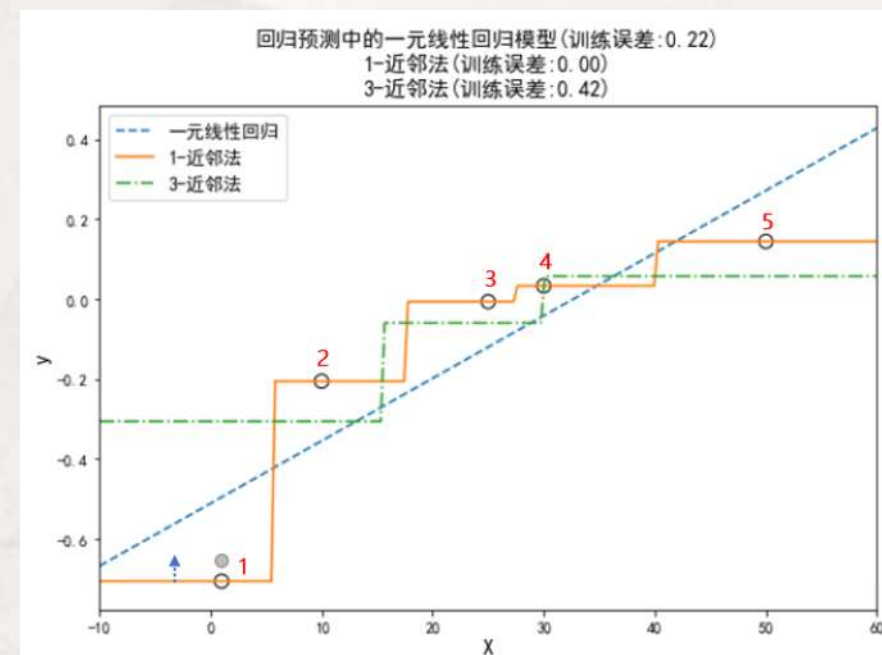


注意：

- 消除数量级的影响
- 一般不直接用于分类型输入变量

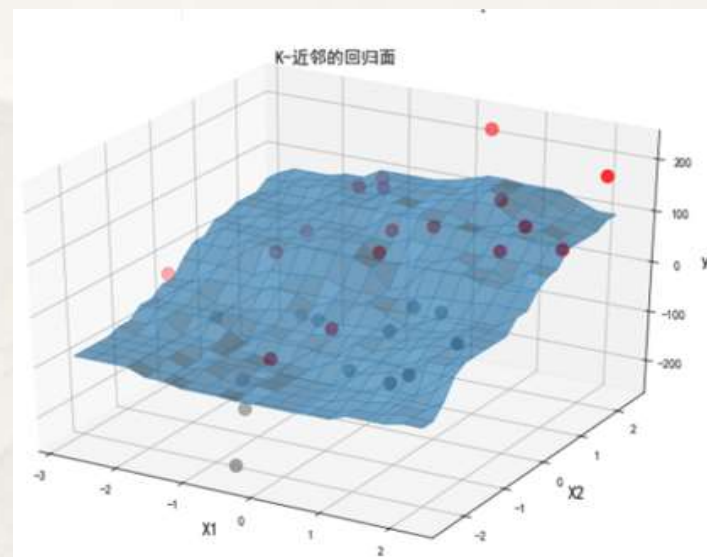
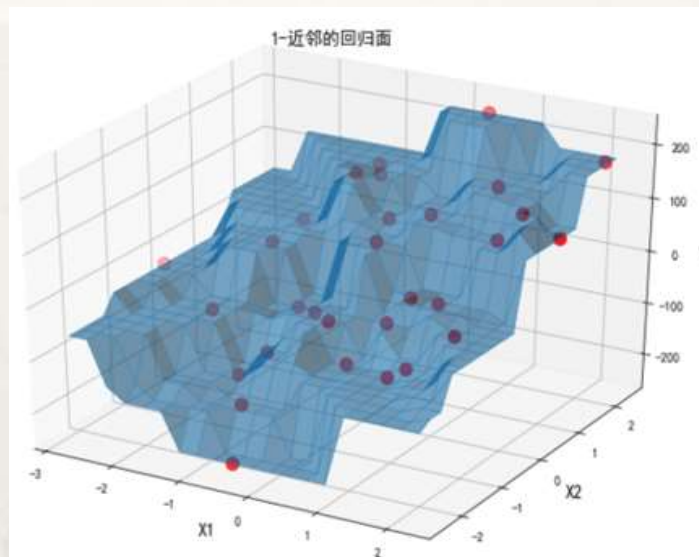
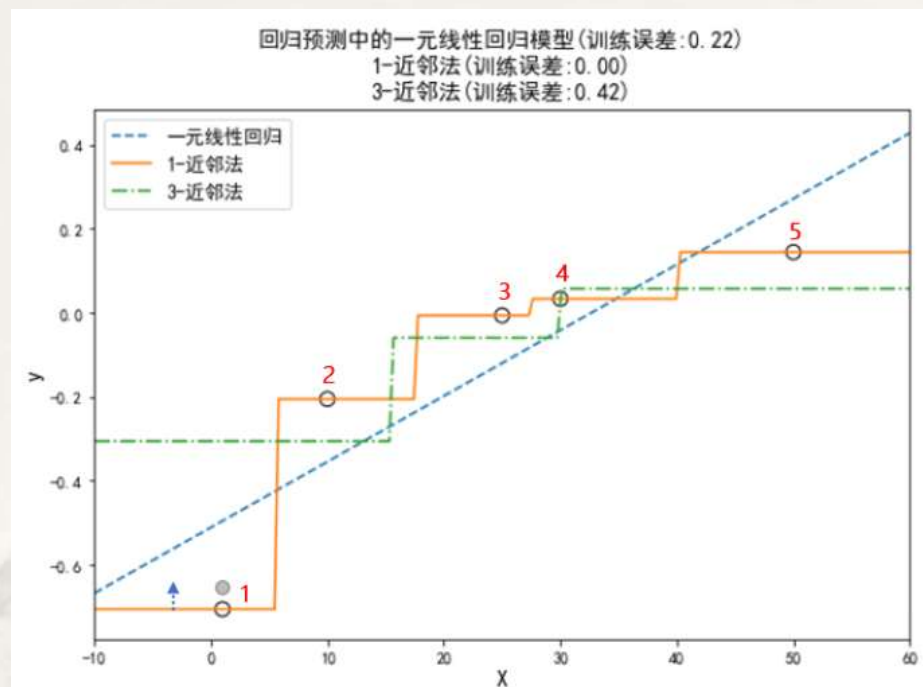
	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- 核心问题2：找到几个近邻：确定K
 - $K = 1$ ：称为1-近邻法，即只找到距离 $\mathbf{X}_0$ 最近的一个近邻 $\mathbf{X}_i$ ，以最近一个近邻的输出变量值作 y_0 的预测值： $\hat{y}_0 = y_i$
 - 问：
 - 1-近邻有什么不足？
 - 如何改进？
 - $K > 1$ ：称为K-近邻法，即找到距离 $\mathbf{X}_0$ 最近的K个近邻($\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_k$)，并根据近邻的输出变量值($y_1, y_2, \dots, y_k$)计算得到 y_0 的预测值
 - 问：
 - 如何确定K？K值大小对预测误差的影响



• 回归预测中的K-近邻法

- Python模拟和启示：**K值大小对预测误差的影响**



- 随着K的增大，误差逐渐增高

线性回归模型的训练误差：0.035
1-近邻法的训练误差：0.000
K-近邻法的训练误差：0.187

- 回归预测中的K-近邻法

- 案例二的分析：一般线性回归模型→K-近邻法：K值大小对预测误差的影响

```
1 X = data[['temp', 'atemp', 'humidity', 'windspeed']]
2 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=1)
3 for Y in ['casual', 'registered']:
4     y = data[Y]
5     modelKNN.fit(X, y)
6     print('\n对%s的预测: R方=%f;MAE=%f' % (Y, modelKNN.score(X, y), mean_absolute_error(y, modelKNN.predict(X))))
```

对casual的预测: R方=0.516859;MAE=14.046114

对registered的预测: R方=0.200637;MAE=66.381040

```
1 X = data[['temp', 'atemp', 'humidity', 'windspeed']]
2 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=5)
3 for Y in ['casual', 'registered']:
4     y = data[Y]
5     modelKNN.fit(X, y)
6     print('\n对%s的预测: R方=%f;MAE=%f' % (Y, modelKNN.score(X, y), mean_absolute_error(y, modelKNN.predict(X))))
```

对casual的预测: R方=0.532203;MAE=20.601966

对registered的预测: R方=0.358369;MAE=86.512034

- 随着K的增大，误差逐渐增高

- 核心问题3：如何确定 X_0 近邻 X_i 的权重

- 权重值的大小取决于 X_i 与 X_0 的相似性
- 核心思想：将相似性定义为 X_i 与 X_0 距离的某种非线性函数。距离越近 X_i 与 X_0 的相似性越强，权重值越高，预测时的影响越大

- 回归预测：
$$\hat{y}_0 = \frac{1}{\sum w_i} (\sum_{i=1}^K w_i y_i)$$

- 权重设计的基本思路

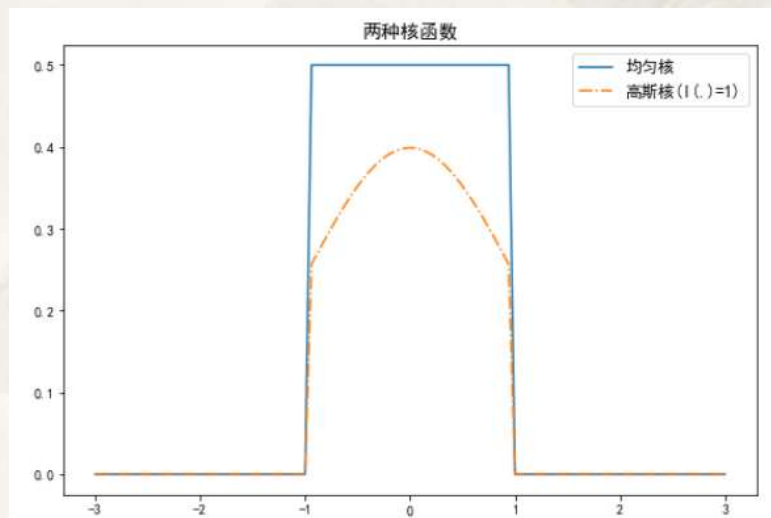
- 设 X_i 与 X_0 的距离为 d_{0i} , ($d_{0i} \geq 0, d_{0i} \in \mathbb{R}$)。函数 $K(d_{0i})$ 可将距离 d_{0i} 转换为 X_i 与 X_0 的相似性，有三个主要特性：
 - 第一， $K(d_{0i}) \geq 0$;
 - 第二， $d_{0i} = 0$ 时， $K(d_{0i})$ 取最大值，即零距离时相似性最大
 - 第三， $K(d_{0i})$ 是 d_{0i} 的单调减函数，即距离越大相似性越小
- 例如： d_{0i} 的倒数
- 例如：核函数

- 权重设计中的核函数

- 核函数是关于样本观测距离的一类函数的总称
- 常用的核函数有：均匀核函数和高斯核函数

- 设函数 $I(d_{0i})$ 为示性函数，只有0和1两个值：
$$I(d_{0i}) = \begin{cases} 1, & |d_{0i}| \leq 1 \\ 0, & |d_{0i}| > 1 \end{cases}$$

$$K(d_{0i}) = \frac{1}{2} \cdot I(|d_{0i}| \leq 1) \quad K(d_{0i}) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{d_{0i}^2}{2}\right) \cdot I(|d_{0i}| \leq 1)$$



- $|d_{0i}| \leq 1$ 时：
 - 均匀核函数：无论 d_{0i} 值大或小， X_i 与 X_0 的相似性都等于0.5，即为等权重
 - 高斯核函数： X_i 与 X_0 的相似性是 d_{0i} 的非线性单调减函数；
- $|d_{0i}| > 1$ 时：均匀核函数和高斯核函数定义的相似性都等于0，意味着此时的 X_i 不是 X_0 的近邻
- 如何保证 X_0 的 K 个近邻与 X_0 的距离 $|d_{0i}| \leq 1$ ？

$$D_{0i} = \frac{d(X_i, X_0)}{d(X_{(K+1)}, X_0)}, i = 1, 2, \dots, K \quad w_i = K(D_{0i})$$

- 案例二的分析：一般线性回归模型→K-近邻法：倒数加权的作用

```
1 X = data[['temp', 'atemp', 'humidity', 'windspeed']]
2 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=5)
3 for Y in ['casual', 'registered']:
4     y = data[Y]
5     modelKNN.fit(X, y)
6     print('\n对%s的预测: R方=%f;MAE=%f'%(Y, modelKNN.score(X, y), mean_absolute_error(y, modelKNN.predict(X))))
```

对casual的预测: R方=0.532203;MAE=20.601966

对registered的预测: R方=0.358369;MAE=86.512034

```
1 X = data[['temp', 'atemp', 'humidity', 'windspeed']]
2 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=5, weights='distance')
3 for Y in ['casual', 'registered']:
4     y = data[Y]
5     modelKNN.fit(X, y)
6     print('\n对%s的预测: R方=%f;MAE=%f'%(Y, modelKNN.score(X, y), mean_absolute_error(y, modelKNN.predict(X))))
```

对casual的预测: R方=0.748561;MAE=12.615926

对registered的预测: R方=0.596212;MAE=57.552897

- 问：本案例目前的遗憾是什么？
 - 分类型变量的处理

- 有能力的同学可拓展：数值型和分类型变量混合下的观测间距离的计算
 - 将数值型变量和分类型变量分开计算
 - 数值型变量：先标准化处理，再计算欧氏(或其他)距离
 - 分类型变量：计算汉明距离(Hamming distance)，即两个样本观测在多少个分类变量上取值不同；或计算杰卡德距离(Jaccard Distance)
 - 通过加权合并为一个总距离
 - 找到合理的权重

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

有序目标编码 (Ordered Target Encoding)

CatBoost (Categorical Boosting)

核心思想

- 对每个分类特征的每个取值，计算其对应的目标变量统计量（如均值），转换为数值。
- 关键改进：避免目标泄漏 (Target Leakage)。传统目标编码使用全局数据，易导致过拟合。

编码步骤

1. 随机排列数据集：训练前，对样本进行随机排序生成一个排列顺序。
2. 动态计算编码：
 - 对于第 i 个样本的某个分类特征，仅使用排列中该样本之前的所有样本（历史数据）计算编码。
 - 公式示例（二分类）：
3. 多排列增强：训练中生成多个随机排列，提升稳定性。

$$\text{编码值} = \frac{\text{历史样本中相同类别对应的目标值之和} + \text{先验参数}}{\text{历史样本中相同类别的数量} + \text{平滑参数}}$$

- 先验参数（如全局目标均值）和平滑参数（如权重）用于减少噪声。

案例三：空气质量等级的预测---多分类预测

- 案例三：根据污染物浓度对空气质量等级进行预测：分类预测

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0
3	2014-01-04	149.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0

质量等级

良	827
轻度污染	470
优	377
中度污染	252
重度污染	127
严重污染	43

- 案例三的分析：softmax回归

```

1 X = data[['PM2.5', 'PM10', 'SO2', 'CO', 'NO2', 'O3']]
2 y = data['质量等级']
3 modelLR2 = LM.LogisticRegression(multi_class='multinomial')
4 modelLR2.fit(X, y)
5 print('总正确率', accuracy_score(y, modelLR2.predict(X)))
6 print('评价模型结果: \n', classification_report(y, modelLR2.predict(X)))

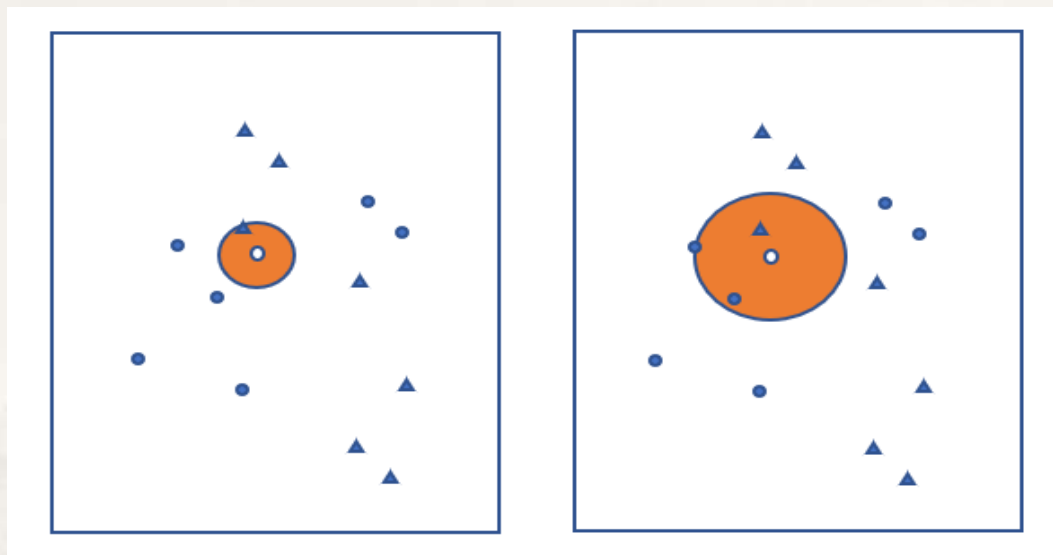
```

总正确率 0.5734732824427481
评价模型结果：

	precision	recall	f1-score	support
严重污染	0.69	0.67	0.68	43
中度污染	0.48	0.35	0.41	252
优	0.68	0.55	0.61	377
良	0.61	0.70	0.65	827
轻度污染	0.43	0.44	0.44	470
重度污染	0.65	0.69	0.67	127
accuracy			0.57	2096
macro avg	0.59	0.57	0.58	2096
weighted avg	0.57	0.57	0.57	2096

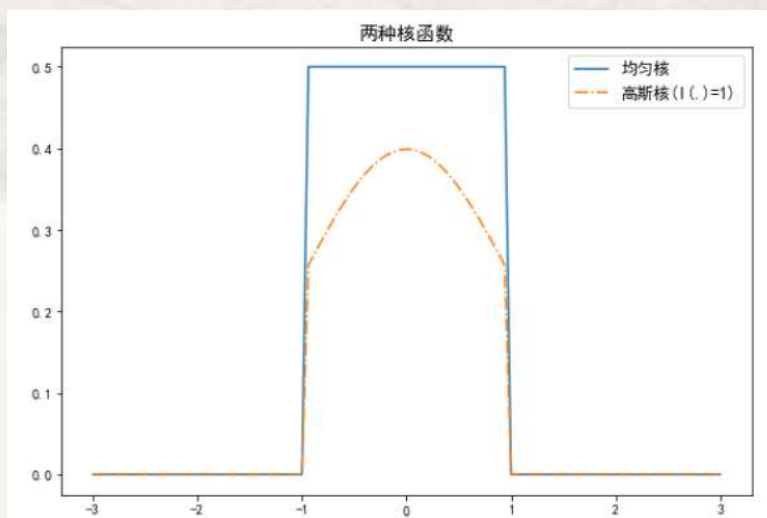
- 问：如何进一步提高预测精度？K-近邻法

- 分类预测中的K-近邻法



分类预测:

$$\hat{y}_0 = f(y_1, y_2, \dots, y_K)$$
$$= \text{mode}(y_1, y_2, \dots, y_K)$$



$$K(d_{0i}) = \frac{1}{2} \cdot I(|d_{0i}| \leq 1)$$

$$K(d_{0i}) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{d_{0i}^2}{2}\right) \cdot I(|d_{0i}| \leq 1)$$

$$\hat{y}_0 = \arg \max_C \sum_{i=1}^K w_i I(y_i = C),$$

soft-max投票/hard-max投票

• 案例三的分析: softmax回归→K-近邻法

```
data = pd.read_excel('北京市空气质量数据.xlsx')
data = data.replace(0,np.NaN)
data = data.dropna()
X = data[['PM2.5','PM10','SO2','CO','NO2','O3']]
y = data['质量等级']
modelKNN = neighbors.KNeighborsClassifier(n_neighbors=5,weights='distance')
modelKNN.fit(X,y)
print('\nK-近邻的总错判率:%f'%(1-modelKNN.score(X,y)))
confusion_matrix(y,modelKNN.predict(X))
```

K-近邻的总错判率:0.000000

```
array([[ 43,   0,   0,   0,   0,   0],
       [  0, 252,   0,   0,   0,   0],
       [  0,   0, 377,   0,   0,   0],
       [  0,   0,   0, 827,   0,   0],
       [  0,   0,   0,   0, 470,   0],
       [  0,   0,   0,   0,   0, 127]], dtype=int64)
```

Softmax回归:

总正确率 0.5734732824427481

评价模型结果:

	precision	recall	f1-score	support
严重污染	0.69	0.67	0.68	43
中度污染	0.48	0.35	0.41	252
优	0.68	0.55	0.61	377
良	0.61	0.70	0.65	827
轻度污染	0.43	0.44	0.44	470
重度污染	0.65	0.69	0.67	127
accuracy			0.57	2096
macro avg	0.59	0.57	0.58	2096
weighted avg	0.57	0.57	0.57	2096

• 问:

- 与Softmax回归相比, K-近邻法可降低误差的原因?

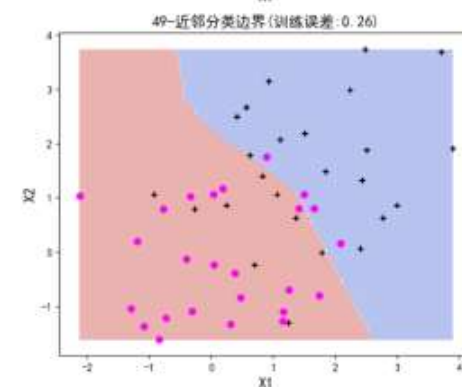
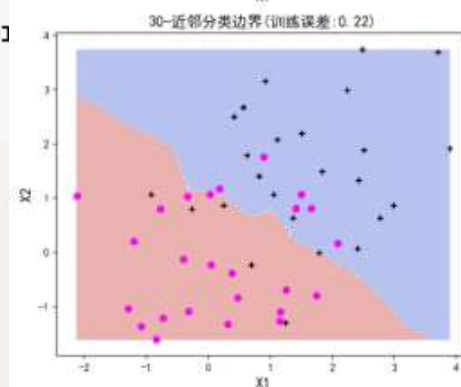
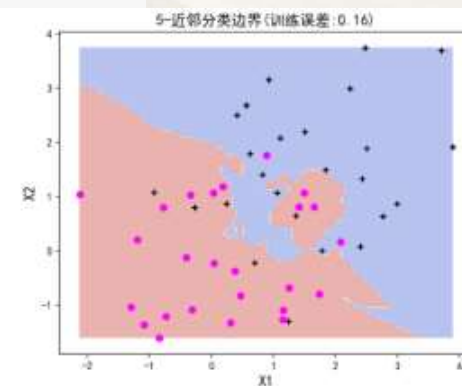
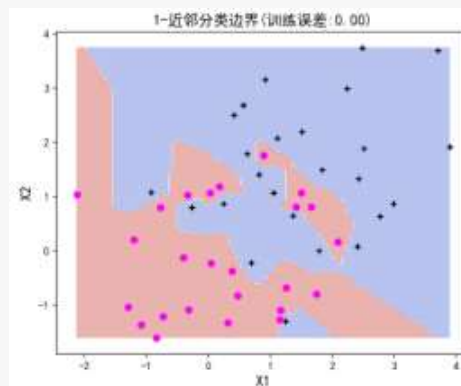
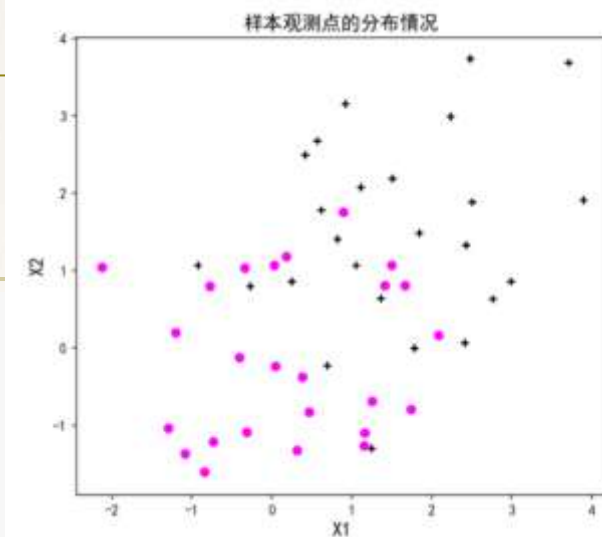
- 分类预测中的K-近邻法——非线性建模
 - Python模拟和启示：认识K-近邻的分类边界

```

1 np.random.seed(123)
2 N = 50
3 n = int(0.5*N)
4 X = np.random.normal(0,1,size=100).reshape(N,2)
5 Y = [0]*n + [1]*n
6 X[0:n] = X[0:n] + 1.5
7 X01,X02 = np.meshgrid(np.linspace(X[:,0].min(),X[:,0].max(),100), np.linspace(X[:,1].min(),X[:,1].max(),100))
8 X0 = np.hstack((X01.reshape(10000,1),X02.reshape(10000,1)))
9 fig,axes = plt.subplots(nrows=2,ncols=2,figsize=(15,12),dpi=150)
10 for K,H,L in [(1,0,0), (5,0,1), (30,1,0), (49,1,1)]:
11     modelKNN = neighbors.KNeighborsClassifier(n_neighbors=K)
12     modelKNN.fit(X,Y)
13     YOhat = modelKNN.predict(X0).reshape(X01.shape)
14     axes[H,L].contourf(X01, X02, YOhat, alpha=0.4, cmap=plt.cm.coolwarm)
15     axes[H,L].scatter(X[:,0],X[:,1],color='black',marker='+')
16     axes[H,L].scatter(X[n:N,0],X[n:N,1],color='magenta',marker='o')
17     axes[H,L].set_title('%d-近邻分类边界(训练误差:%.2f)'%(K,1-modelKNN.score))
18     axes[H,L].set_xlabel('X1',fontsize = 14)
19     axes[H,L].set_ylabel('X2',fontsize = 14)

```

Chapter5-1.ipynb



- 非线性分类边界
- 随着K由小增大，误差由小变大
 - 问：还得到哪些启示？

- 分类预测中的K-近邻法——非线性建模
 - Python模拟和启示：认识K-近邻的分类边界

- 1-近邻法

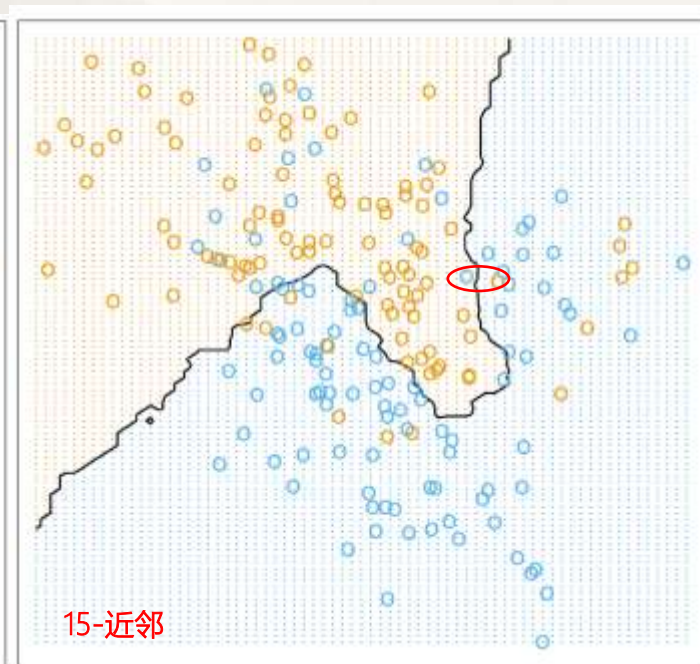
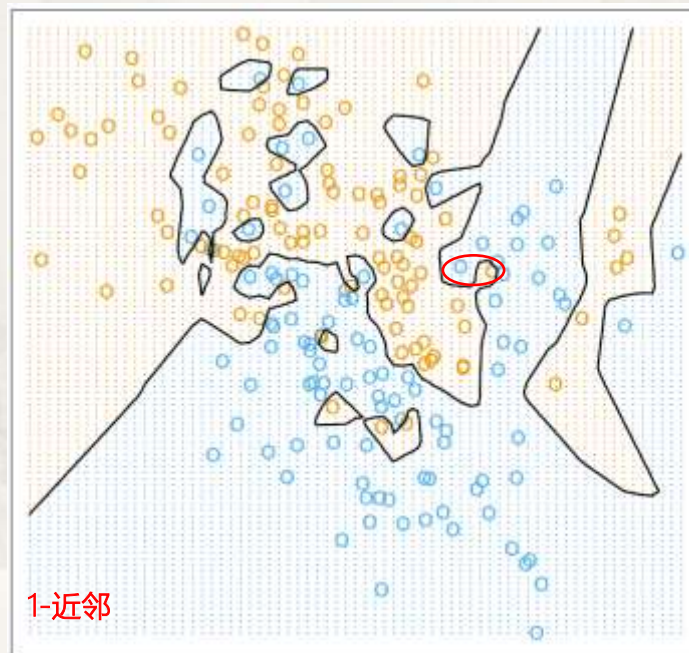
- 优势：特别适合分类边界极不规则的情况
 - 不足：预测不稳定
 - 问：圈的颜色会变？

- K-近邻法

- 有效弥补1-近邻法的不足
 - 分类边界趋于规则

- 问：

- K-近邻法的观测权重设计在分类预测中的作用？



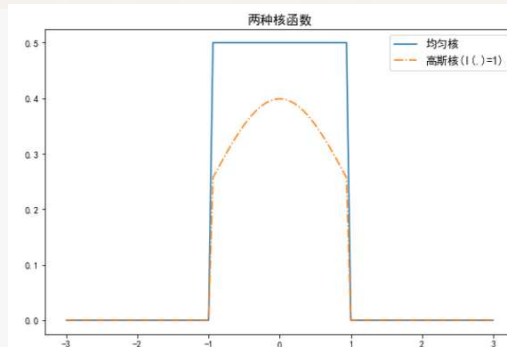
Python模拟和启示：认识加权K-近邻的分类边界

Chapter5-3.ipynb

```
def guass(x):
    x = preprocessing.scale(x, axis=1)
    output = 1/np.sqrt(2*np.pi)*np.exp(-x*x/2)
    return output

np.random.seed(123)
N = 50
n = int(0.5*N)
X = np.random.normal(0, 1, size=100).reshape(N, 2)
Y = [0]*n+[1]*n
X[0:n] = X[0:n]+1.5
X01, X02 = np.meshgrid(np.linspace(X[:,0].min(), X[:,0].max(), 100), np.linspace(X[:,1].min(), X[:,1].max(), 100))
X0 = np.hstack((X01.reshape(10000, 1), X02.reshape(10000, 1)))
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 12))
for W, K, H, L, T in [('distance', 5, 0, 0, '倒数加权'), (guass, 5, 0, 1, '高斯加权'),
                      ('distance', 10, 1, 0, '倒数加权'), (guass, 10, 1, 1, '高斯加权')]:
    modelKNN = neighbors.KNeighborsClassifier(n_neighbors=K, weights=W)
    modelKNN.fit(X, Y)
    YOhat = modelKNN.predict(X0)
    YOhat = YOhat.reshape(X01.shape)
    axes[H, L].contourf(X01, X02, YOhat, alpha=0.4, cmap=plt.cm.coolwarm)
    axes[H, L].scatter(X[:,0], X[:,1], color='black', marker='x')
    axes[H, L].scatter(X[n:N, 0], X[n:N, 1], color='magenta', marker='o')
    axes[H, L].set_title('%d-近邻分类边界-%s(训练误差%.2f)'%(K, T, 1-modelKNN.score(X, Y)))
```

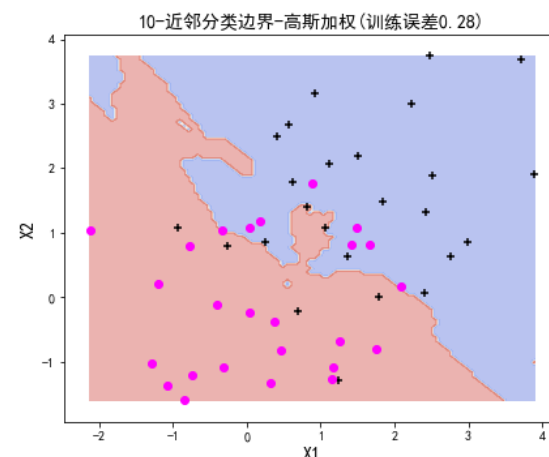
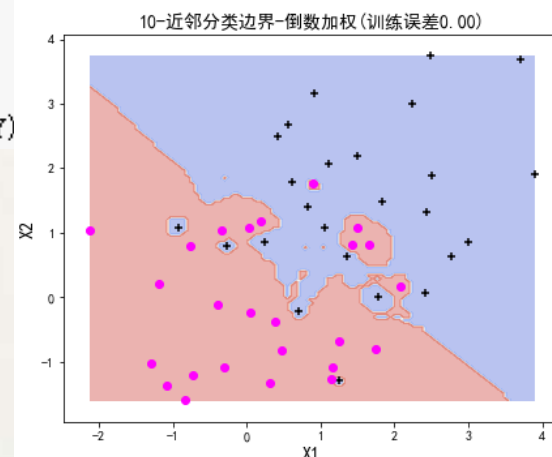
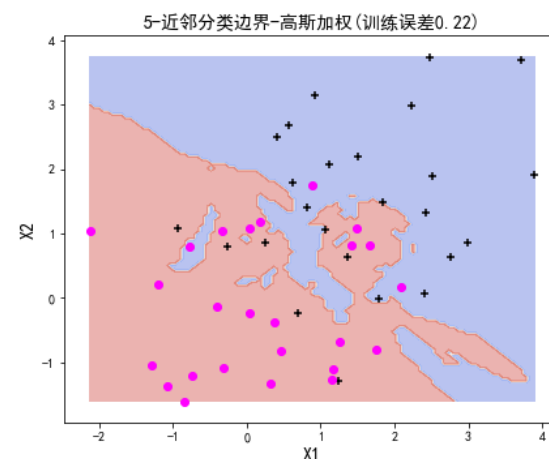
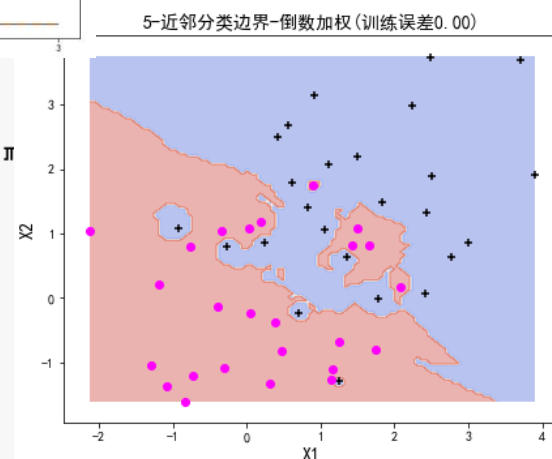
- 本例中加权K-近邻法的预测效果与未加权的1-近邻法持平
- 能有效克服1-近邻法预测不稳定的问题



$$K(d_{oi}) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{d_{oi}^2}{2}\right) \cdot I(|d_{oi}| \leq 1)$$

$$\hat{y}_0 = \arg \max_C \sum_{i=1}^K w_i I(y_i = C),$$

soft-max投票/hard-max投票



- 案例三的分析：softmax回归→K-近邻法

```
data = pd.read_excel('北京市空气质量数据.xlsx')
data = data.replace(0,np.NaN)
data = data.dropna()
X = data[['PM2.5','PM10','SO2','CO','NO2','O3']]
y = data['质量等级']
modelKNN = neighbors.KNeighborsClassifier(n_neighbors=5,weights='distance')
modelKNN.fit(X,y)
print('\nK-近邻的总错判率:%f'%(1-modelKNN.score(X,y)))
confusion_matrix(y,modelKNN.predict(X))
```

K-近邻的总错判率:0.000000

```
array([[ 43,   0,   0,   0,   0,   0],
       [   0, 252,   0,   0,   0,   0],
       [   0,   0, 377,   0,   0,   0],
       [   0,   0,   0, 827,   0,   0],
       [   0,   0,   0,   0, 470,   0],
       [   0,   0,   0,   0,   0, 127]], dtype=int64)
```

- 问：

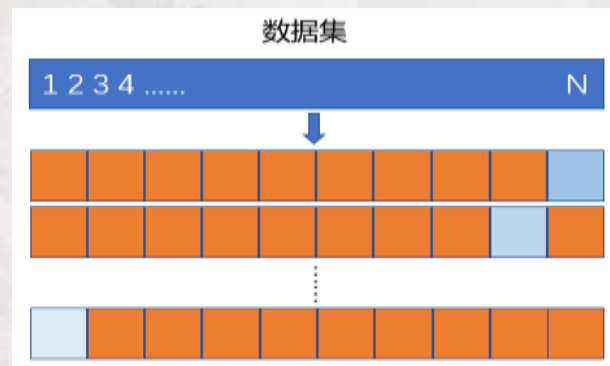
- 案例中基于K-近邻法的预测误差等于0，完美！？

案例引申出的问题1：需评估模型在**未来的**预测表现，如何**评估**？

- 引入新概念：**训练误差和泛化误差**，两者的不同点：
 - 针对**袋内观测**和**袋外观测**(Out Of Bag, **OOB**)计算的误差
 - 问**：什么是袋外观测？
 - 可直接计算和无法直接计算(只能给出**估计**，如何**估计**？)
 - 方法：数据集的划分---训练集和测试集



旁置法



K折(10折)交叉验证法(适合样本量较小的情况)

- 用于分别计算**训练误差**和**测试误差**
 - 测试误差，因针对OOB可作为泛化误差的估计

```
CVscore = cross_val_score(modelLR, X, y, cv=k, scoring='neg_mean_squared_error') #sklearn.metrics.SCORERS.keys()
print("k=10折交叉验证的测试MSE:", -1*CVscore.mean())
scores = cross_validate(modelLR, X, y, scoring='neg_mean_squared_error', cv=k, return_train_score=True)
print("k=10折交叉验证的测试MSE:", -1*scores['test_score'].mean()) # scores为字典
```

问：训练误差和测试误差还有其他**不同的**重要特征吗？

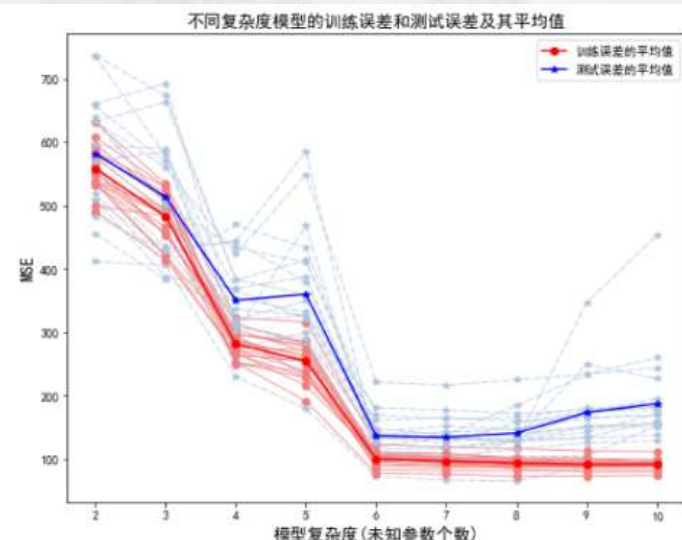
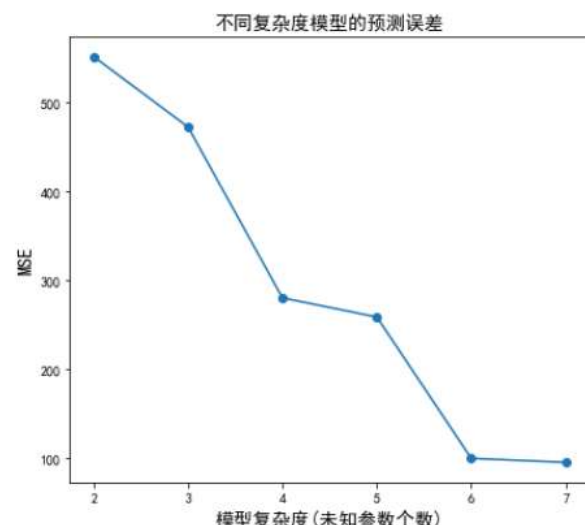
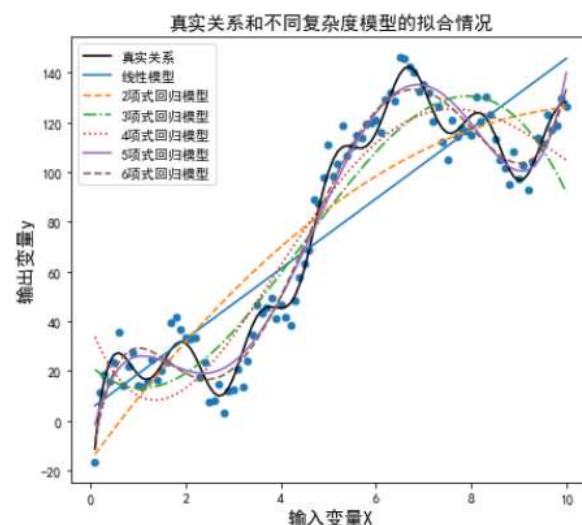
案例引申出的问题2：训练误差与测试误差的重要不同点

Python模拟和启示：

```

1 plt.figure(figsize=(9,7))
2 modellar = LM.LinearRegression()
3 MSEtrain = []
4 MSEtest = []
5 np.random.seed(123)
6 for j in range(20):
7     x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.70)
8     Ntraining = len(y_train)
9     Ntest = len(y_test)
10    msetrain = []
11    msetest = []
12    X_train = x_train.reshape(Ntraining,1)
13    X_test = x_test.reshape(Ntest,1)
14    modellar = LM.LinearRegression()
15    for i in np.arange(1,10):
16        modellar.fit(X_train, y_train)
17        msetrain.append(np.sum((y_train-modellar.predict(X_train))**2)/(Ntraining-(i+1)))
18        msetest.append(np.sum((y_test-modellar.predict(X_test))**2)/(Ntest-(i+1)))
19        tmp = pow(x_train,i+1).reshape(Ntraining,1)
20        X_train = np.hstack((X_train,tmp))
21        tmp = pow(x_test,i+1).reshape(Ntest,1)
22        X_test = np.hstack((X_test,tmp))
23    plt.plot(np.arange(2,11),msetrain,marker='o',linewidth=0.8,c='lightcoral',linestyle='-')
24    plt.plot(np.arange(2,11),msetest,marker='*',linewidth=0.8,c='lightsteelblue',linestyle='-')
25    MSEtrain.append(msetrain)
26    MSEtest.append(msetest)
27    plt.plot(np.arange(2,11),np.mean(MSEtrain,0),marker='o',linewidth=1.5,c='red',linestyle='-',label="训练误差的平均值")
28    plt.plot(np.arange(2,11),np.mean(MSEtest,0),marker='*',linewidth=1.5,c='blue',linestyle='-',label="测试误差的平均值")
29    plt.title("不同复杂度模型的训练误差和测试误差及其平均值",fontsize=14)
30    plt.xlabel("模型复杂度(未知参数个数)",fontsize=14)
31    plt.ylabel("MSE",fontsize=14)
32    plt.legend()
    
```

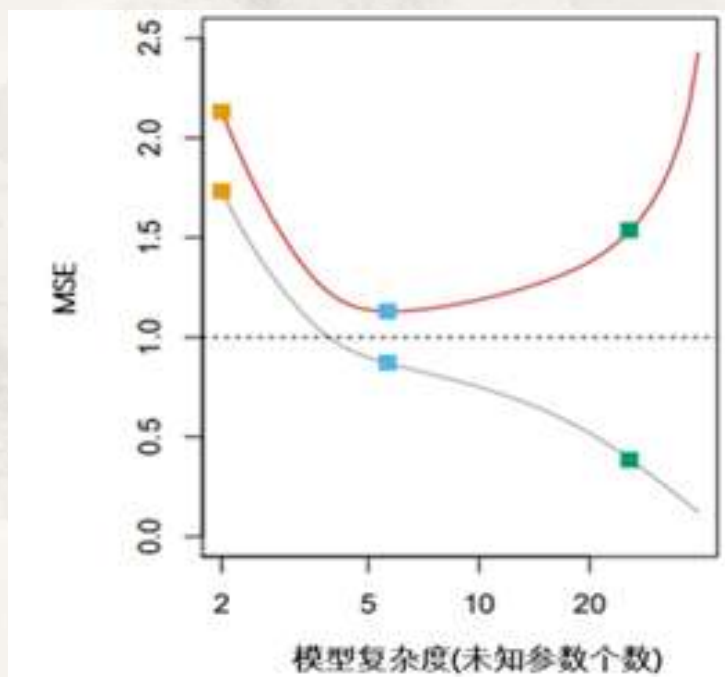
Chapter3-2-1.ipynb



问：

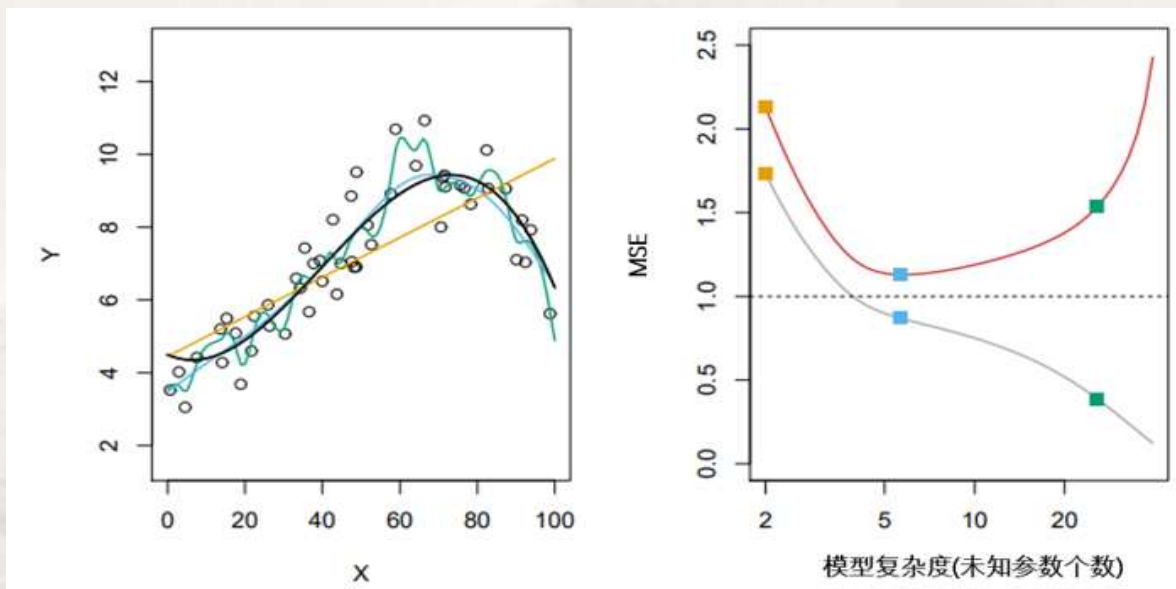
- 本例是用什么测度模型的复杂度？
- 两条不同颜色的曲线有什么特点？

- 案例引申出的问题2：训练误差与测试误差的重要不同点
 - 随模型复杂度增加训练误差和测试误差曲线呈不同的变化趋势
 - U字形的测试误差曲线
 - 训练误差最小时测试误差可能不是最小的



- 问：
 - 了解训练误差与测试误差的不同点，有什么意义？

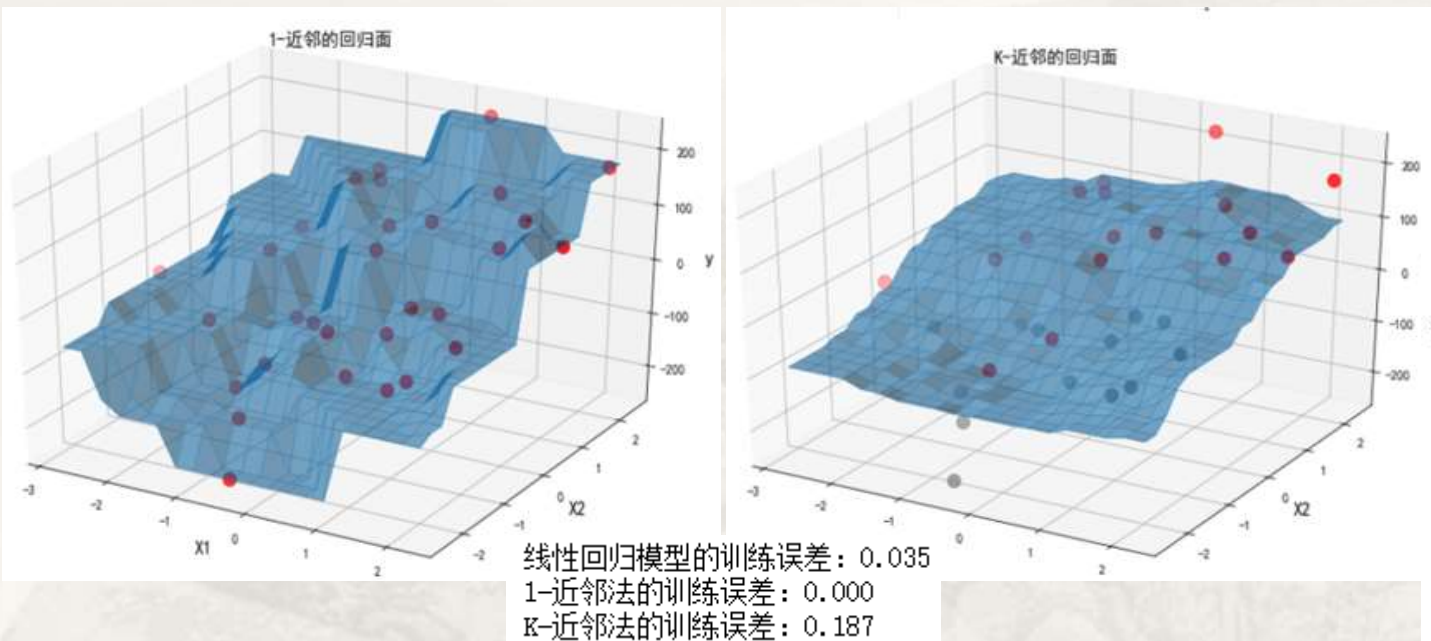
- **案例引申出的问题3：训练误差与测试误差的不同点有什么意义？**
 - 预测模型的训练误差较小但测试误差较大，是模型**过拟合**的重要表现之一，是判断模型是否**过拟合**的重要**工具**
 - 什么是模型的**过拟合**？



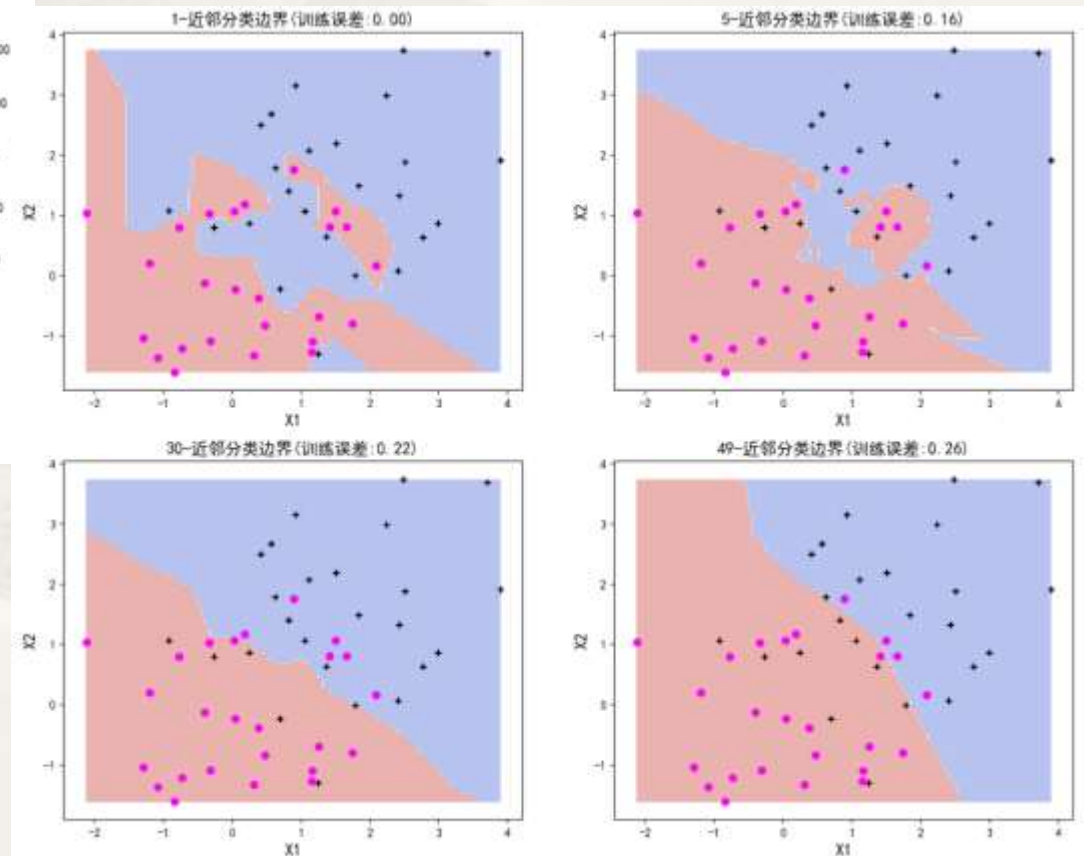
- 以训练误差最小为目标估计模型参数，可能导致预测模型远远偏离输入变量和输出变量的真实关系，从而在新数据集上有较大预测误差

- **问：**
 - 怎样的**模型易出现过拟合(欠拟合)**？上图可见：**模型复杂度**
 - 为什么线性回归分析中**不过于强调**模型的过拟合问题？**模型复杂度**
 - K-近邻分析中模型复杂度的**表现形态是怎样的**？

- 案例引申出的问题4：模型复杂度的直观表现形式和测度(参数)
 - K-近邻法不同复杂度的表现形态：Python模拟和启示
 - 回归曲面或分类边界的平滑程度，体现了模型的复杂度



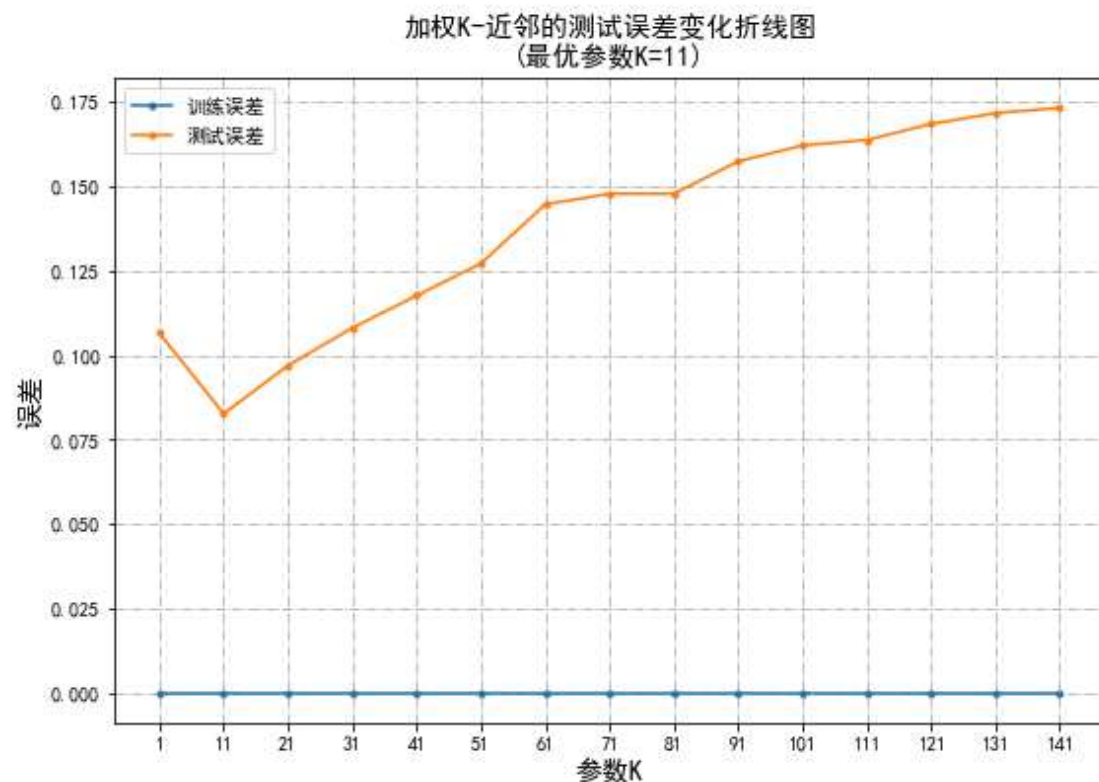
- 随K由小增大，回归曲面越趋于平滑
- 随K由小增大，分类边界越趋于平滑
- 随K由小增大，模型复杂度由高到低，训练误差由小到大



- 案例三的分析：softmax回归→K-近邻法
 - 案例三的K-近邻法出现过拟合问题了吗？

```
trainErr = []
testErr = []
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.70, random_state = 123)
Ntrain = len(y_train)
K = np.arange(1, int(Ntrain*0.10), 10)
for k in K:
    modelKNN = neighbors.KNeighborsClassifier(n_neighbors=k, weights='distance')
    modelKNN.fit(X_train, y_train)
    trainErr.append(1-modelKNN.score(X_train, y_train))
    testErr.append(1-modelKNN.score(X_test, y_test))
bestK = K[testErr.index(np.min(testErr))]
plt.figure(figsize = (9,6))
plt.plot(K, trainErr, marker = '.', label = '训练误差')
plt.plot(K, testErr, marker = '.', label = '测试误差')
plt.title('加权K-近邻的测试误差变化折线图\n(最优参数K=%d)' % bestK, fontsize=14)
plt.grid(True, linestyle = '-.')
plt.xticks(K)
plt.xlabel('参数K', fontsize = 14)
plt.ylabel('误差', fontsize = 14)
plt.legend()
modelKNN = neighbors.KNeighborsClassifier(n_neighbors = bestK, weights = 'distance')
modelKNN.fit(X_train, y_train)
print('模型评价(测试集): \n', classification_report(y_test, modelKNN.predict(X_test)))
```

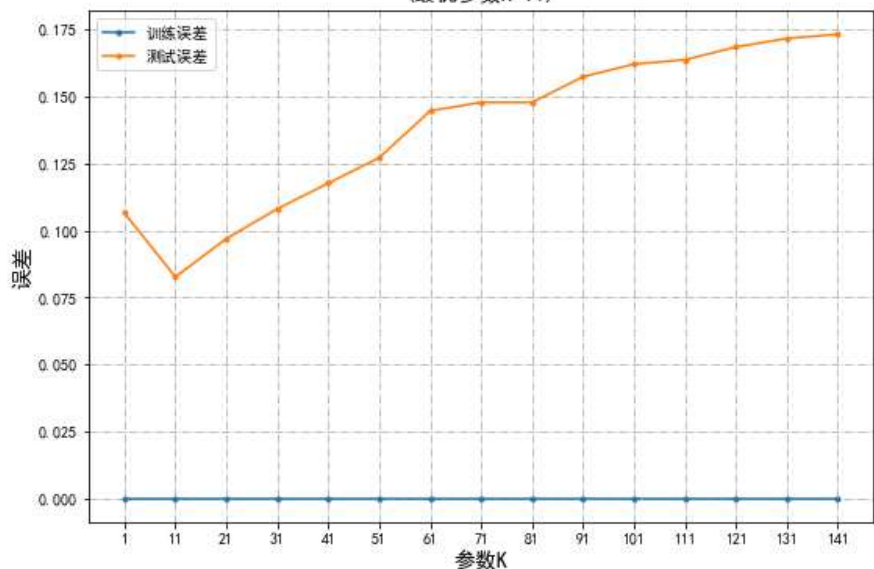
- 问：
 - 应如何观察上图？
 - 上图表明了什麼？可以如何调整？



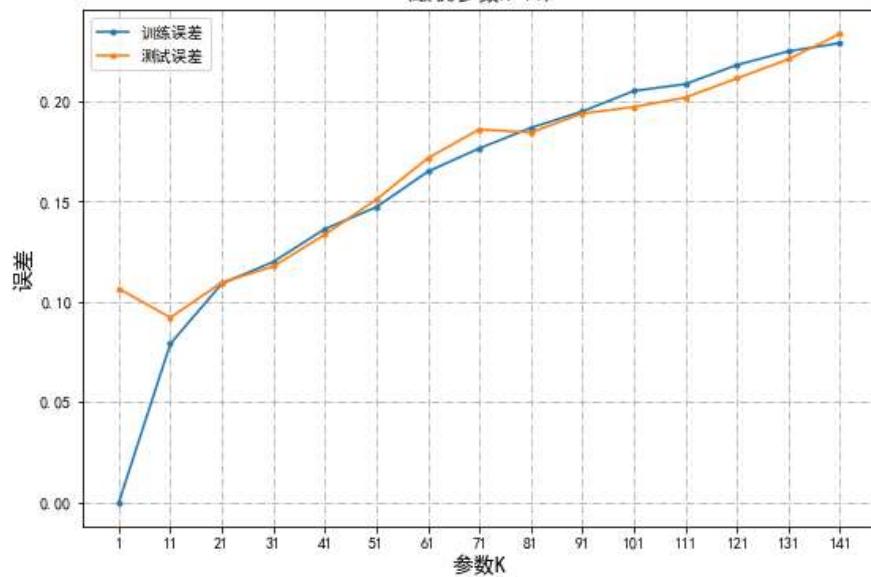
- 案例三的分析：softmax回归→K-近邻法
 - 案例三的K-近邻法出现过拟合问题了吗？

`modelKNN = neighbors.KNeighborsClassifier(n_neighbors = k, weights='uniform')`

加权K-近邻的测试误差变化折线图
(最优参数K=11)



等权K-近邻的测试误差变化折线图
(最优参数K=11)



模型评价(测试集):

	precision	recall	f1-score	support
严重污染	0.94	0.94	0.94	16
中度污染	0.82	0.83	0.82	70
优	0.98	0.93	0.95	133
良	0.93	0.97	0.95	229
轻度污染	0.90	0.90	0.90	144
重度污染	0.93	0.76	0.84	37
accuracy			0.92	629
macro avg	0.91	0.89	0.90	629
weighted avg	0.92	0.92	0.92	629

模型评价(测试集):

	precision	recall	f1-score	support
严重污染	1.00	0.94	0.97	16
中度污染	0.80	0.80	0.80	70
优	0.97	0.93	0.95	133
良	0.92	0.96	0.94	229
轻度污染	0.88	0.89	0.88	144
重度污染	0.93	0.76	0.84	37
accuracy			0.91	629
macro avg	0.92	0.88	0.90	629
weighted avg	0.91	0.91	0.91	629

问题与思考:

- 倾向选择哪个？
- 复杂度适中的模型可避免过拟合，在预测中还有什么优势？

- 案例引申出的问题5：复杂度适中的模型预测中有什么优势？

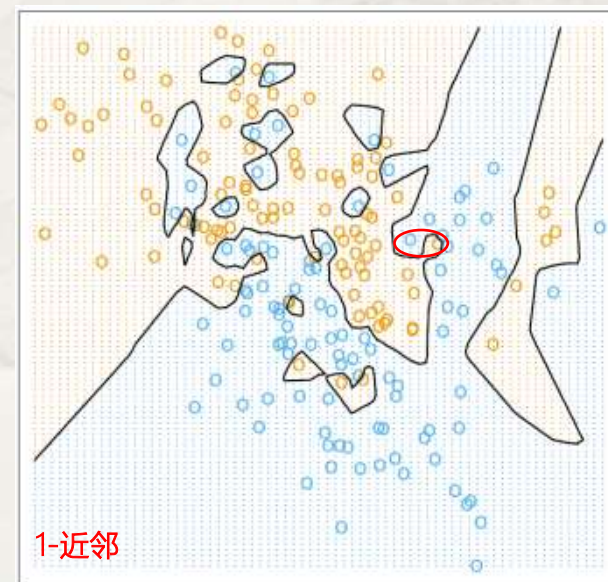
- 具有一定的预测稳健性和较低的预测偏差

- 什么是预测稳健性？

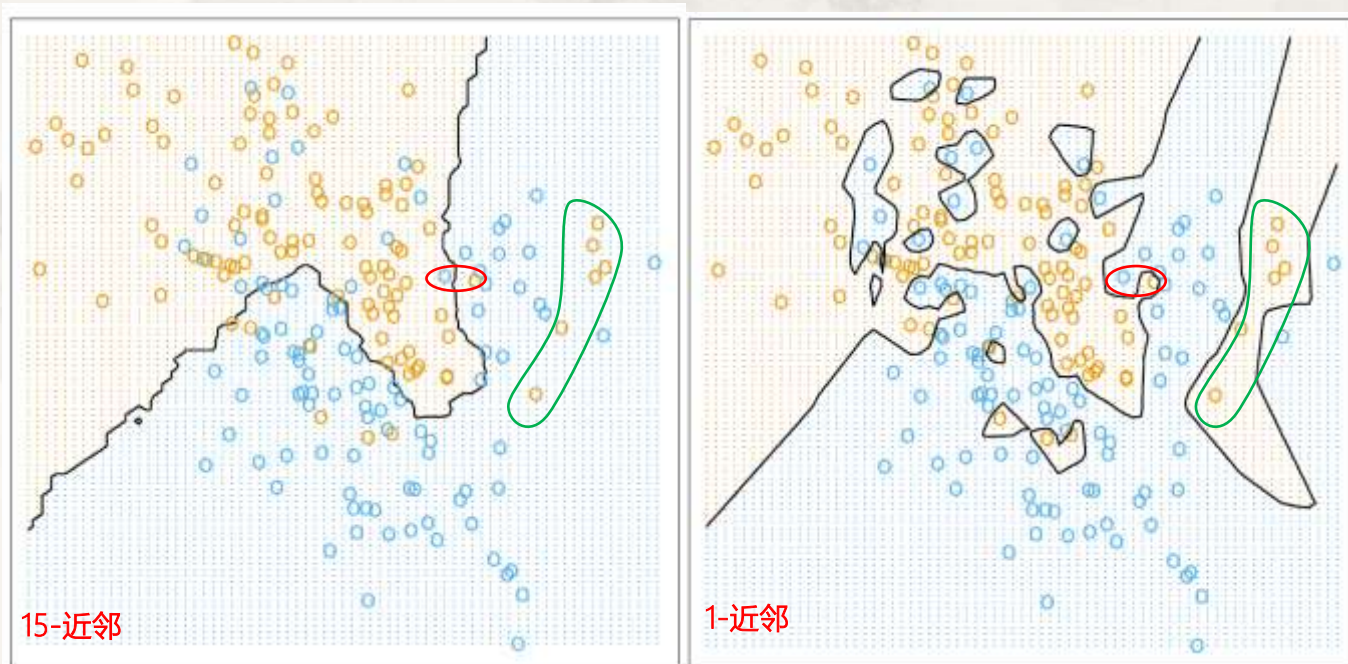
- 指对新数据集中的样本观测 X_0 (OOB)，模型给出的多个预测值 $\hat{y}_0^i, (i = 1, 2, \dots)$ 的波动性小——预测模型的方差
- 理论上，预测方差越小越好，既表明模型具有鲁棒性(稳健性)，也保证了所给的泛化误差估计具有可靠性

- 问：

- 为什么对样本观测 X_0 会有多个预测值？
- 怎样的模型预测稳健性高(方差小)？



- 案例引申出的问题5：复杂度适中的模型预测中有什么优势？
 - 具有一定的预测稳健性和较低的预测偏差
 - 问：怎样的模型预测稳健性高(方差小)？



- 低复杂度的模型稳健性高!
 - 请举一个复杂度最低的Logistic回归的例子
- 但：
 - 低复杂度的模型会有怎样的问题？
 - 预测误差和偏差

• 案例引申出的问题5：复杂度适中的模型预测中有什么优势？

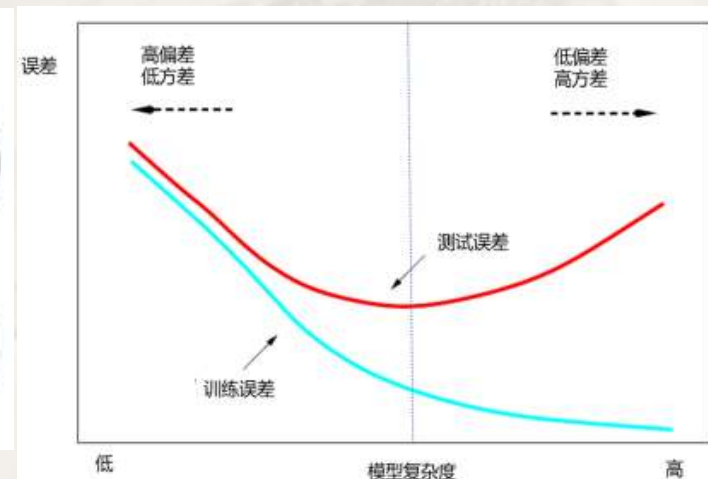
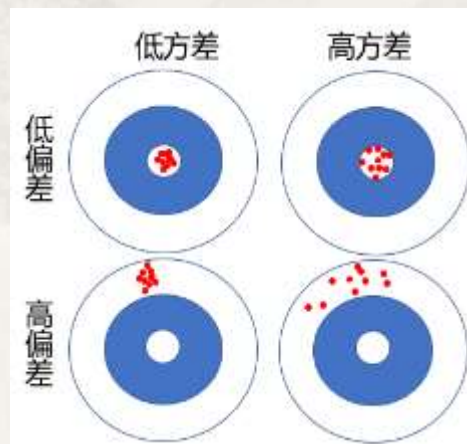
- 具有一定的预测稳健性和较低的预测偏差

• 什么是预测模型的偏差？

- 指对新数据集中的样本观测 X_0 ，模型给出的多个预测值 $\hat{y}_0^i (i = 1, 2, \dots)$ 的期望 $E(\hat{y}_0^i)$ ，与其实值 y_0 的差
- 理论上，预测模型的偏差越小越好，表明模型泛化误差小

• 问：

- 理想中的模型是右图中的哪个？
- 怎样的模型预测偏差小？
 - 高复杂度的模型偏差小但方差大
 - 预测模型的偏差和方差不可兼得！
- 为什么是测试误差曲线，为什么呈U字型？

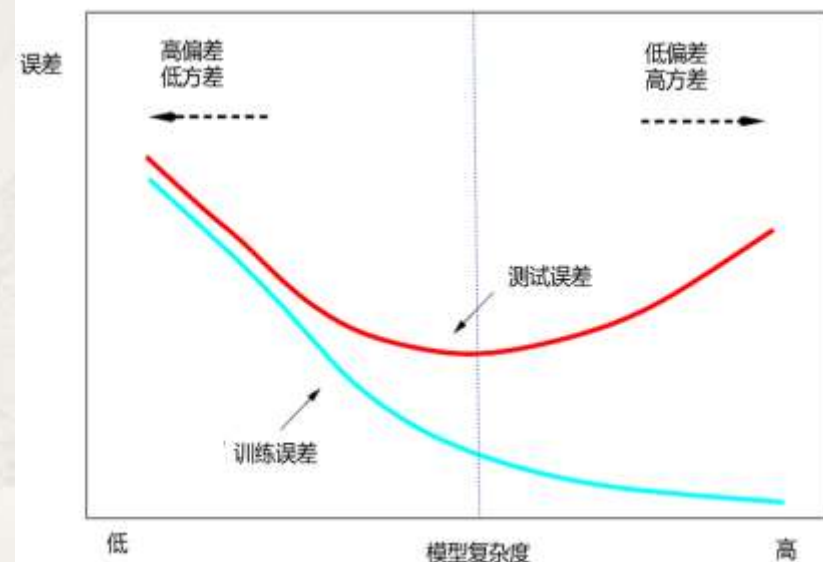


- 案例引申出的问题5：复杂度适中的模型预测中有什么优势？

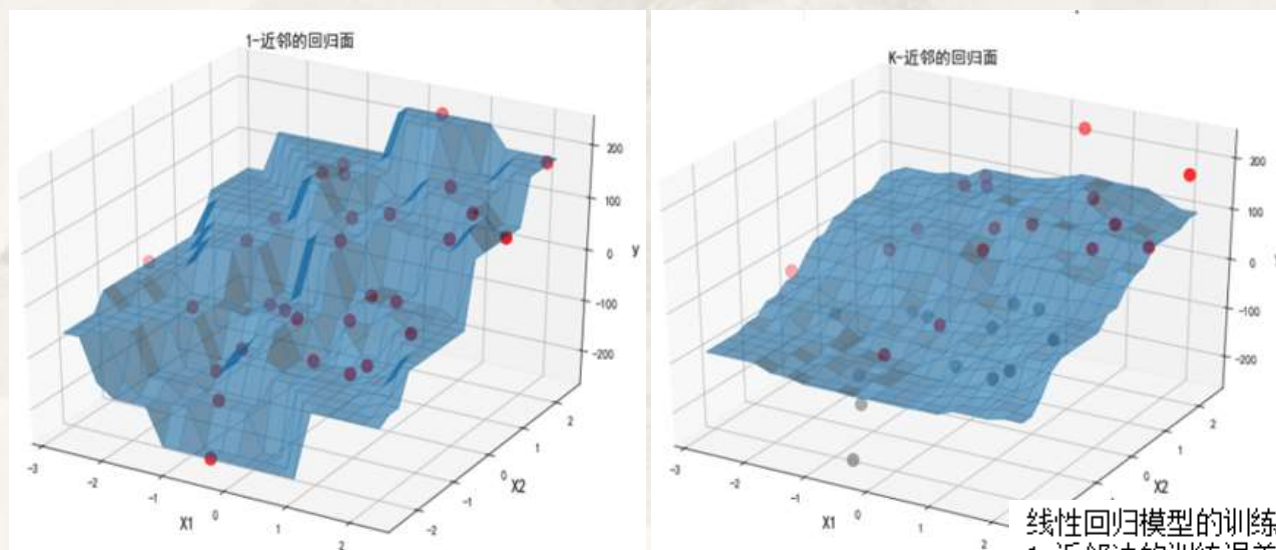
- 问：为什么是测试误差曲线，为什么呈U字型？
 - 理论上的讨论：预测模型的偏差-方差分解

$$\begin{aligned} \text{Err}(\mathbf{X}_0) &= E[(y_0 - \hat{y}_0)^2] = E[f(\mathbf{X}_0) + \epsilon - \hat{f}(\mathbf{X}_0)]^2 = E[f(\mathbf{X}_0) - \hat{f}(\mathbf{X}_0)]^2 + \text{Var}(\epsilon) \\ &\quad \text{OOB} \\ &= [E\hat{f}(\mathbf{X}_0) - f(\mathbf{X}_0)]^2 + E[\hat{f}(\mathbf{X}_0) - E\hat{f}(\mathbf{X}_0)]^2 + \text{Var}(\epsilon) \\ &= [\text{Bias}(\hat{y}_0)]^2 + \text{Var}(\hat{y}_0) + \text{Var}(\epsilon) \end{aligned}$$

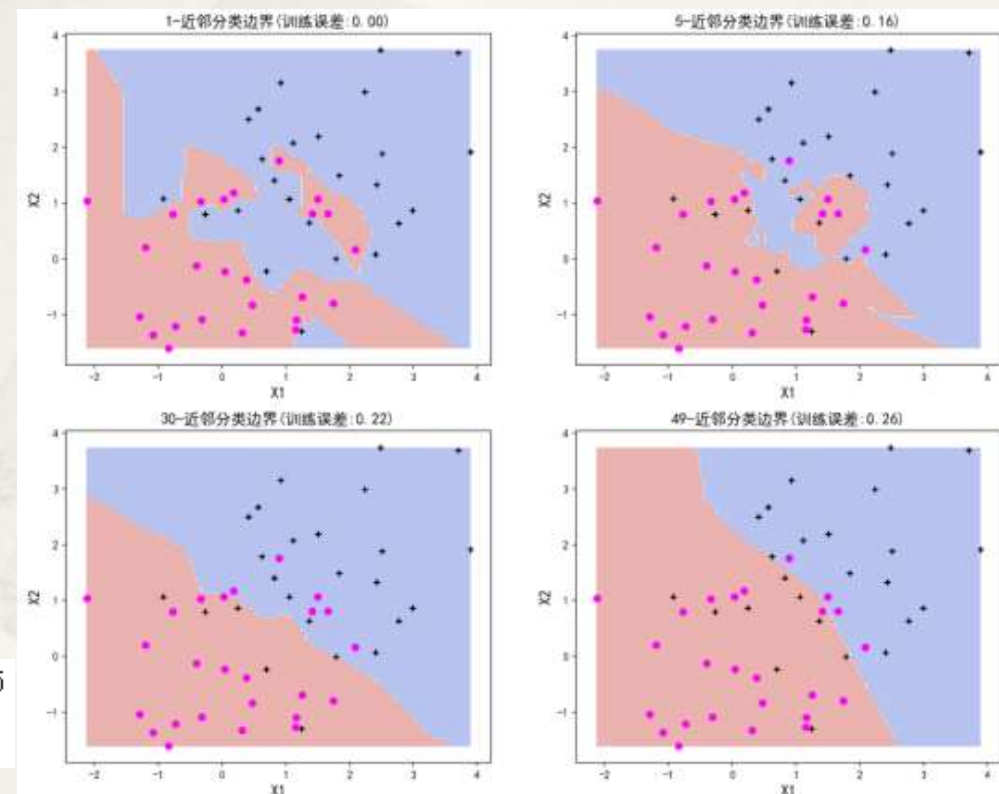
- 偏差的减少可以抵消方差的增加， $\text{Err}(\mathbf{X}_0)$ 会继续降低
- 偏差的减少无法抵消方差的增加， $\text{Err}(\mathbf{X}_0)$ 则会增加
- 复杂度适中模型的优势：兼顾了偏差和方差的平衡
 - 理想模型！
- 问：
 - 案例三的K-近邻法，可通过怎样的途径得到复杂度适中的模型？



- 案例三的分析： **softmax回归** → **K-近邻法**
 - 可通过怎样的**途径**得到**复杂度适中**的模型：**超参数调优**
 - 什么是**超参数**？人为指定，不是通过模型训练得到的参数
- 问：
 - K-近邻法的**超参数**？
 - 人为指定超参数的**取值**，意味着什么？



线性回归模型的训练误差：0.035
 1-近邻法的训练误差：0.000
 K-近邻法的训练误差：0.187



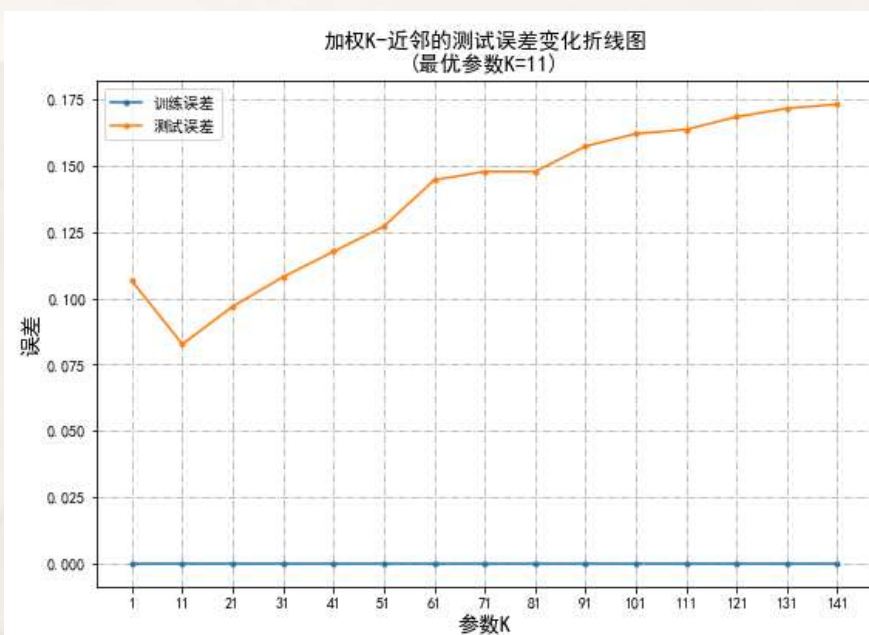
- 如何进行超参数调优？

- 案例三的分析： softmax回归→K-近邻法
 - 可通过怎样的途径得到复杂度适中的模型： 超参数调优

```
trainErr = []
testErr = []
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.70, random_state = 123)
Ntrain = len(y_train)
K = np.arange(1, int(Ntrain*0.10), 10)
for k in K:
    modelKNN = neighbors.KNeighborsClassifier(n_neighbors=k, weights='distance')
    modelKNN.fit(X_train, y_train)
    trainErr.append(1-modelKNN.score(X_train, y_train))
    testErr.append(1-modelKNN.score(X_test, y_test))
bestK = K[testErr.index(np.min(testErr))]
plt.figure(figsize = (9,6))
plt.plot(K, trainErr, marker = '.', label = '训练误差')
plt.plot(K, testErr, marker = '.', label = '测试误差')
plt.title('加权K-近邻的测试误差变化折线图\n(最优参数K=%d)' % bestK, fontsize=14)
plt.grid(True, linestyle = '-.')
plt.xticks(K)
plt.xlabel('参数K', fontsize = 14)
plt.ylabel('误差', fontsize = 14)
plt.legend()
modelKNN = neighbors.KNeighborsClassifier(n_neighbors = bestK, weights = 'distance')
modelKNN.fit(X_train, y_train)
print('模型评价(测试集): \n', classification_report(y_test, modelKNN.predict(X_test)))
```

问：

- 测试集的用途？
- 测试误差(1 - 0.92)可作为泛化误差的估计吗？



模型评价(测试集):

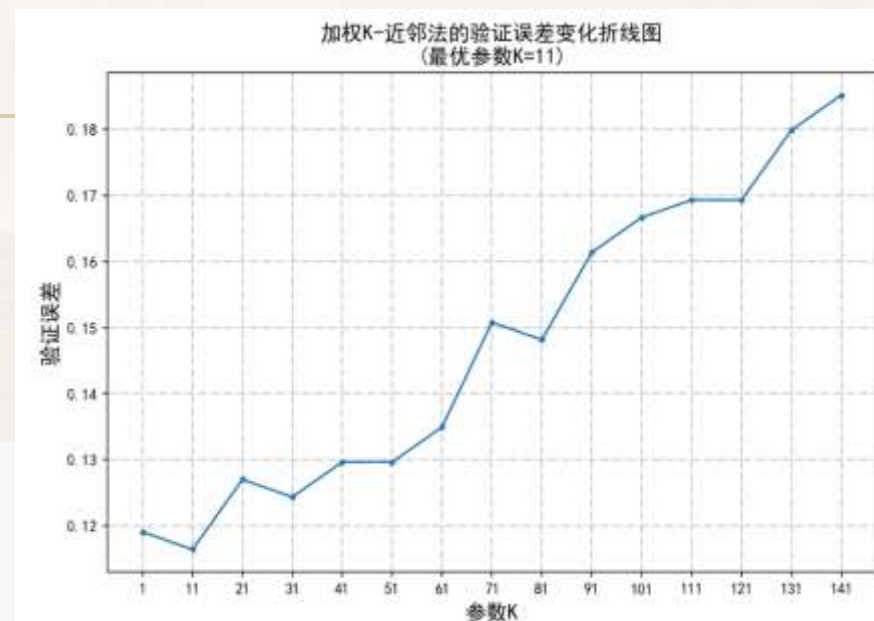
	precision	recall	f1-score	support
严重污染	0.94	0.94	0.94	16
中度污染	0.82	0.83	0.82	70
优	0.98	0.93	0.95	133
良	0.93	0.97	0.95	229
轻度污染	0.90	0.90	0.90	144
重度污染	0.93	0.76	0.84	37
accuracy			0.92	629
macro avg	0.91	0.89	0.90	629
weighted avg	0.92	0.92	0.92	629

- 案例三的分析： softmax回归→K-近邻法
- 问：测试集可用来估计模型的泛化误差吗？
 - 数据集三划分策略：训练集、测试集和验证集
 - 数据泄露！测试集的数据信息泄露给了训练集
 - 用于调参的数据集称为验证集(Validation Set)—验证误差
 - 找到验证误差最小时的K

```

6 valErr = []
7 X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
8 X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, train_size = 0.80, random_state = 123)
9 Ntrain = len(y_train)
10 K = np.arange(1, int(Ntrain*0.10), 10)
11 for k in K:
12     modelKNN = neighbors.KNeighborsClassifier(n_neighbors = k, weights='distance')
13     modelKNN.fit(X_train, y_train)
14     valErr.append(1-modelKNN.score(X_val, y_val))
15 bestK = K[valErr.index(np.min(valErr))]
16 plt.figure(figsize = (9,6), dpi = 150)
17 plt.plot(K, valErr, marker='.')
18 plt.title('加权K-近邻法的验证误差变化折线图\n(最优参数K=%d)'%bestK, fontsize = 14)
19 plt.grid(True, linestyle = '-.')
20 plt.xticks(K)
21 plt.xlabel('参数K', fontsize = 14)
22 plt.ylabel('验证误差', fontsize = 14)
23 modelKNN=neighbors.KNeighborsClassifier(n_neighbors = bestK, weights = 'distance')
24 modelKNN.fit(X_train, y_train)
25 print('模型评价(测试集): \n', classification_report(y_test, modelKNN.predict(X_test)))

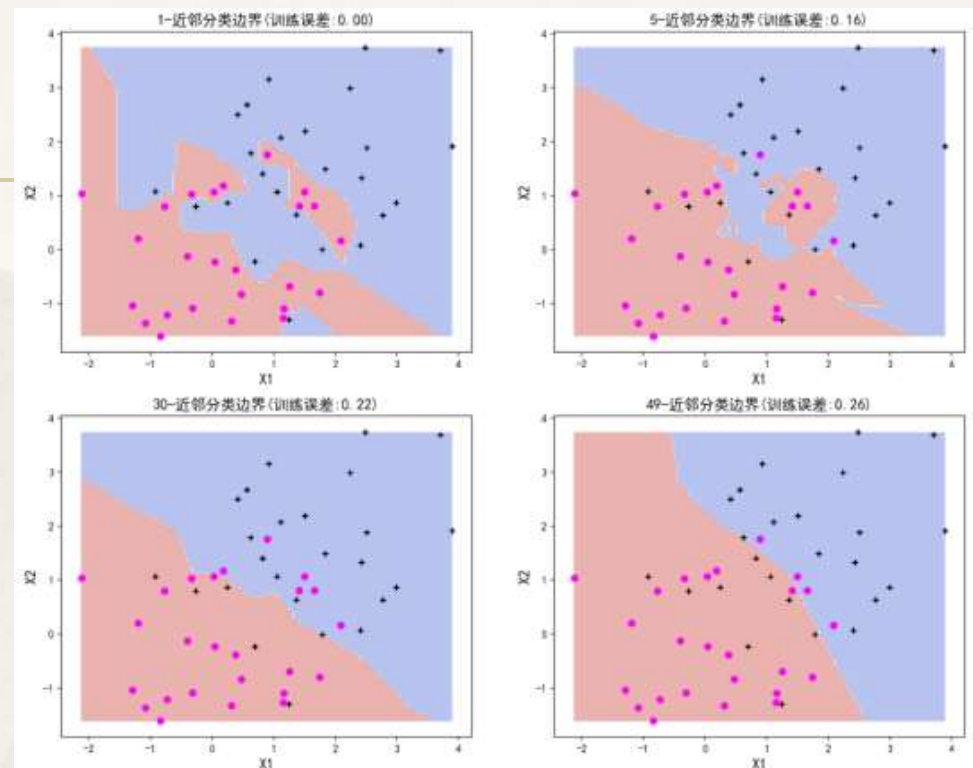
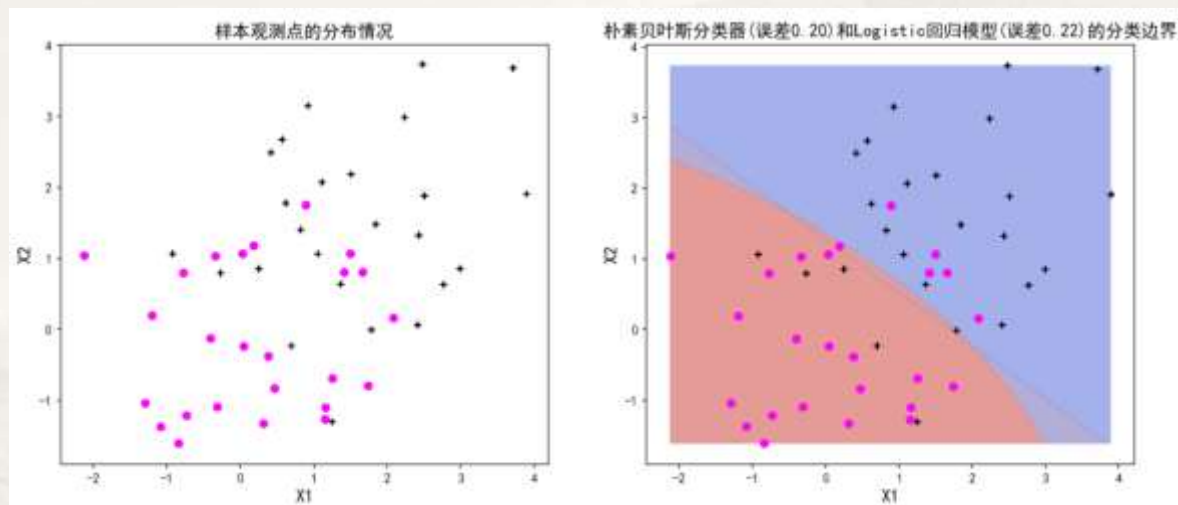
```



模型评价(测试集):

	precision	recall	f1-score	support
严重污染	1.00	0.75	0.86	4
中度污染	0.75	0.75	0.75	24
优	0.97	0.93	0.95	40
良	0.92	0.97	0.94	78
轻度污染	0.87	0.90	0.88	51
重度污染	0.89	0.62	0.73	13
accuracy			0.90	210
macro avg	0.90	0.82	0.85	210
weighted avg	0.90	0.90	0.89	210

- K-近邻法与Logistic回归模型的对比：
 - K-近邻法的模型复杂度更高(K较小时)，更适合解决非线性分类问题



- K-近邻法是一种基于局部的学习
 - Logistic回归：总损失函数是基于训练集全体构造，并由此估计模型参数
 - K-近邻法仅基于部分样本观测直接完成预测，是一种基于以 X_0 为圆心以较短长度为半径的邻域圆(或球)的局部方法(Local Methods)
- K-近邻法是一种“懒惰学习”算法：没有传统意义上的模型“训练”过程
 - 问：可以认为输出变量y没有参与算法？
 - 训练过程；预测过程

- K-近邻法的Python应用实践：基于某段时间国产电视剧的播放和大众评分数据（文件名：电视剧播放数据.xlsx），通过观众给出的点赞数和差评数，对电视剧的得分进行回归预测（仅对点赞数和差评数低于200万以下的电视剧进行分析）

```
1 data=pd.read_excel('电视剧播放数据.xlsx')
2 data=data.replace(0,np.NaN)
3 data=data.dropna()
4 data=data.loc[(data['点赞']<=2000000) & (data['差评']<=2000000)]
5 data.head()
```

	剧名	类型	播放量	点赞	差评	得分
0	花千骨2015	言情剧\n\n穿越剧\n\n网络剧	3.07亿	992342	357808.0	7.3
1	还珠格格2015	古装剧\n\n喜剧\n\n网络剧	73.3万	2352	7240.0	2.5
2	天局	武侠剧\n\n古装剧\n\n悬疑剧\n\n网络剧	3454万	38746	3593.0	9.2
3	明若晓溪	青春剧\n\n言情剧\n\n偶像剧	1.57亿	518660	72508.0	8.8
4	多情江山	言情剧\n\n古装剧\n\n宫廷剧	1126万	22553	6955.0	7.6

Chapter5-5.ipynb

```
6 X = data.loc[:, ['点赞', '差评']]
7 y = data.loc[:, '得分']
8 testErr = []
9 X_train,X_test, y_train, y_test = train_test_split(X,y,train_size=0.70, random_state=123)
10 K = np.arange(1,30,2)
11 for k in K:
12     modelKNN = neighbors.KNeighborsRegressor(n_neighbors=k)
13     modelKNN.fit(X_train,y_train)
14     testErr.append(1-modelKNN.score(X_test,y_test))
15 bestK = K[testErr.index(np.min(testErr))]
16 print('最优参数K:', bestK)
17 modelKNN = neighbors.KNeighborsRegressor(n_neighbors=bestK)
18 modelKNN.fit(X_train,y_train)
19 print('K-近邻：测试误差=%.3f;总预测误差=%.3f'%(1-modelKNN.score(X_test,y_test), 1-modelKNN.score(X,y)))
20 modelLR = LM.LinearRegression()
21 modelLR.fit(X_train,y_train)
22 print('线性回归模型：测试误差=%.3f;总预测误差=%.3f'%(1-modelLR.score(X_test,y_test), 1-modelLR.score(X,y)))
```

最优参数K: 3

K-近邻：测试误差=0.004;总预测误差=0.002

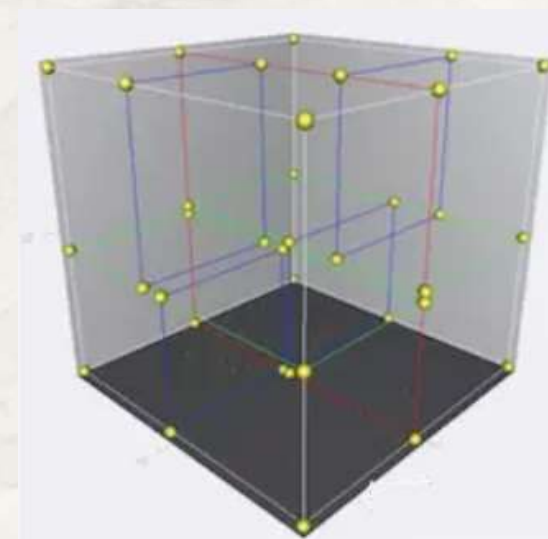
线性回归模型：测试误差=0.820;总预测误差=0.824

- 建立基于K-近邻的回归预测模型
- 建立一般线性回归预测模型
- 对比两个模型的score(拟合优度 R^2)，寻找原因？

K-近邻法的算法思考

- 算法角度的思考

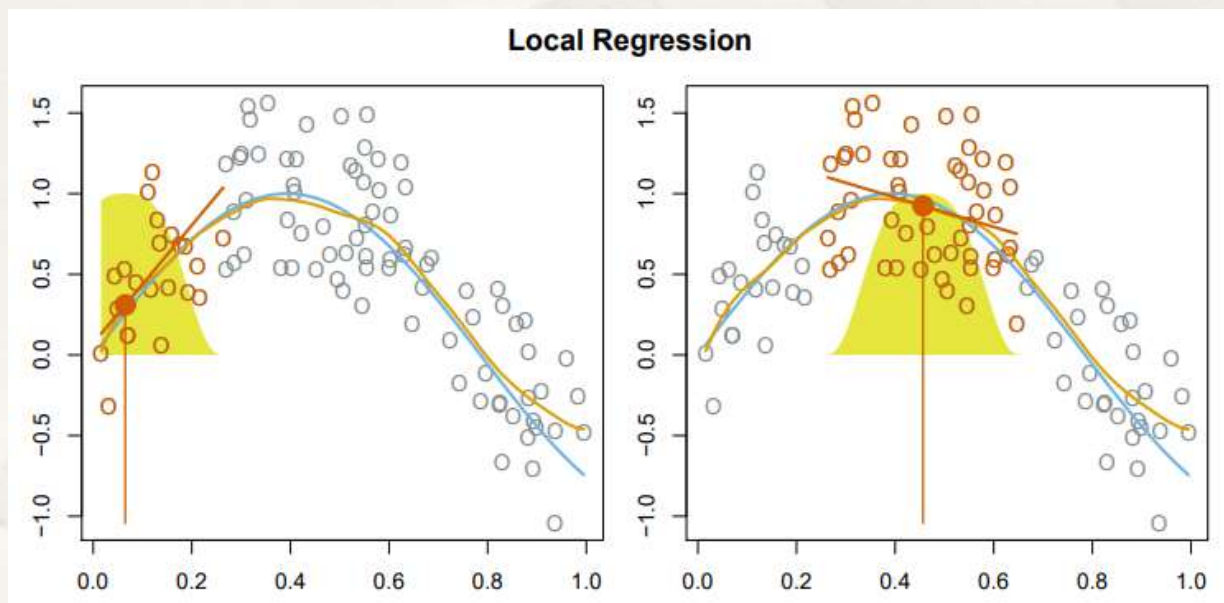
- 最近邻搜索算法：在一个确定的距离度量和一个搜索空间内寻找与给定查询项距离最小的元素
 - 暴力搜索法的时间复杂度 $O(n)$ ：线性阶，数据量增大 n 倍，耗时也增大 n 倍
 - KD tree(K-Dimension tree):
 - 对数据点所在的 K 维空间不断进行二分，从而得到若干个局部空间。最近邻搜索时不断进行分支判断，选择其中的局部子空间，避免全局空间搜索



- 统计角度的思考，不同学科的殊途同归

K-近邻法的统计思考

- 近邻法的统计思考：局部回归(Local Regression)的特例
 - 以一元回归为例：加权的局部平均



- 蓝色曲线表示真实的系统关系 f ；黄色为拟合的 $\hat{f}$
- 橙色点为近邻(局部)
- 钟形表示权重确定方式
- $X = X_0$ 时的加权局部线性回归模型为：

Fit a *weighted least squares regression* of the y_i on the x_i using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

$$\sum_{i=1}^n K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2.$$

The fitted value at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

- 聚焦数据的局部结构！

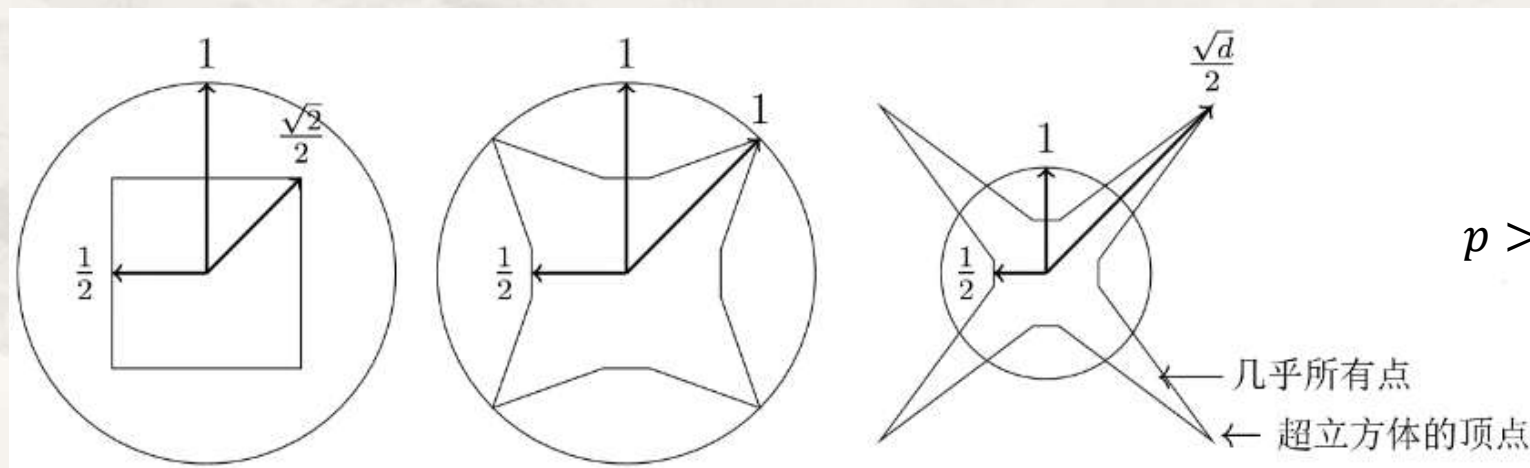
- K-近邻法属于常数(constant)回归

K-近邻法的适用性探讨

- K-近邻法通常适用于输入变量为数值型的情况
 - 对分类型输入变量，K-近邻的处理策略
 - 例如：根据人的性别、身高、体重、体脂率判断是否经常锻炼
 - 分别到男性或女性中依据身高、体重、体脂率寻找近邻
 - 本质：默认性别对分析有重要影响
 - 并非上述情况都成立
 - 例如：依据学生的性别、多门课程的成绩判断实习可否被录用
- 仍需要探索的方面：
 - K-近邻分析中默认各个变量的重要性相同，可能不合理
 - K-近邻分析未给出输入变量的重要性

K-近邻法的适用性探讨

- K-近邻法仅基于部分样本观测直接完成预测，是一种基于以 X_0 为圆心以较短长度为半径的邻域圆（或球）的**局部方法**（Local Methods）
 - 适用于输入变量空间 $\mathbb{R}^p$ **维度较低**的情况。空间维度高导致的问题：
 - 空间维度 p 的增加，K-近邻法基于“邻域”的局部性特征将逐渐丧失，导致预测误差增大

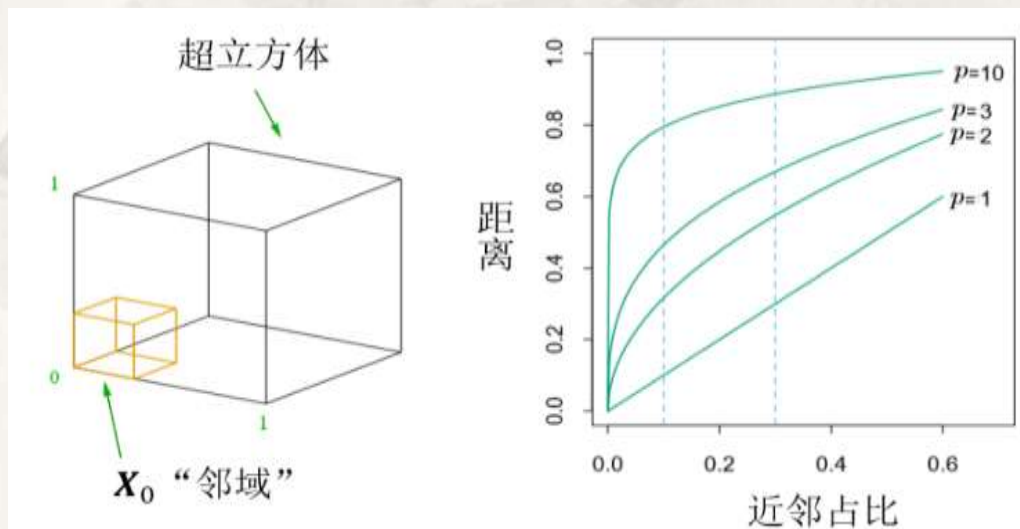


$$p > 4: d(X_v, X_0) = \frac{\sqrt{p}}{2} > 1$$

- X_0 邻域内点的样本量会远远小于K-近邻法的参数 K （即使 K 较小）

K-近邻法的适用性探讨

- K-近邻法仅基于部分样本观测直接完成预测，是一种基于以 X_0 为圆心以较短长度为半径的邻域圆（或球）的**局部方法**（Local Methods）
 - 适用于输入变量空间 $\mathbb{R}^p$ **维度较低**的情况。空间维度高导致的问题：
 - 空间维度 p 的增加，K-近邻法基于“邻域”的局部性特征将逐渐丧失，导致预测误差增大



- 要找到近邻占比等于 r 的 X_0 的近邻，邻域边长的期望为：

$$Ed_p(r) = r^{1/p}$$

r 为近邻占比
 p 为空间维度

$$Ed_{10}(0.1) = 0.1^{1/10} = 0.8$$

$$Ed_{10}(0.01) = 0.01^{1/10} = 0.63$$

- 降维，选择对输出变量预测有重要影响的输入变量非常重要